

File: aoi_controller.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```
import time
from .cnc_controller import GRBLCNCController
from .camera_controller import CameraController
from .position_manager import InspectionPositionManager, InspectionPosition, InspectionSequence

class CNCAOIController:
    """
    Controlador principal que integra o movimento CNC e captura de imagem
    para aplicações de Inspeção Óptica Automatizada (AOI).
    """

    def __init__(self, serial_port=None, camera_interface=None):
        """
        Inicializa o controlador AOI.

        Args:
            serial_port: Porta serial para conexão com a CNC (opcional)
            camera_interface: Interface para a câmera (opcional)
        """
        self.cnc = GRBLCNCController()
        self.camera = CameraController(camera_interface)
        self.position_manager = InspectionPositionManager()
        self.is_running_sequence = False
        self.current_sequence = None
        self.current_position_index = -1

    def connect_cnc(self, port=None, baudrate=115200):
        """Conecta à máquina CNC."""
        return self.cnc.connect(port, baudrate)

    def run_gcode_file(self, filename, callback=None):
        """
        Executa um arquivo G-CODE diretamente

        Args:
            filename: Caminho do arquivo G-CODE
            callback: Função chamada após cada captura
        """
        from aoi_lib.gcode_manager import GCodeManager
        gcode_manager = GCodeManager()

        # Carrega o G-CODE para uma sequência temporária
        gcode_data = gcode_manager.read_gcode_file(filename)

        if not gcode_data:
            raise ValueError(f"Erro ao carregar arquivo G-CODE: {filename}")

        # Cria posições a partir dos dados do G-CODE
```

```

positions = []
for pos_data in gcode_data['positions']:
    position = InspectionPosition(
        pos_data['name'],
        pos_data['x'],
        pos_data['y'],
        pos_data['camera_params']
    )
    positions.append(position)

# Cria sequência temporária
temp_sequence = InspectionSequence(gcode_data['sequence_name'])
for pos in positions:
    temp_sequence.add_position(pos)

# Executa a sequência
return self.run_sequence(temp_sequence.name, callback)

def disconnect_cnc(self):
    """Desconecta da máquina CNC."""
    if self.cnc.is_connected:
        return self.cnc.disconnect()
    return True

def connect_camera(self, camera_id=0):
    """Conecta à câmera."""
    return self.camera.connect(camera_id)

def add_inspection_position(self, name, x, y, camera_params=None):
    """
    Adiciona uma posição de inspeção ao gerenciador.

    Args:
        name: Nome descritivo da posição
        x, y: Coordenadas
        camera_params: Parâmetros específicos da câmera (opcional)
    """
    position = InspectionPosition(name, x, y, camera_params)
    self.position_manager.add_position(position)
    return position

def add_current_position(self, name, camera_params=None):
    """Adiciona a posição atual como uma posição de inspeção."""
    position = self.cnc.get_current_position()
    return self.add_inspection_position(name, position['x'], position['y'], camera_params)

def create_sequence(self, name, positions=None):
    """Cria uma nova sequência de inspeção."""
    sequence = InspectionSequence(name)
    if positions:
        for pos in positions:

```

```
        sequence.add_position(pos)
    self.position_manager.add_sequence(sequence)
    return sequence
```

```
def run_sequence(self, sequence_name, callback=None):
```

```
    """
```

Executa uma sequência de inspeção completa.

Args:

sequence_name: Nome da sequência a executar

callback: Função chamada após cada captura com imagem e metadados

```
    """
```

```
    sequence = self.position_manager.get_sequence(sequence_name)
```

```
    if not sequence:
```

```
        raise ValueError(f"Sequência '{sequence_name}' não encontrada")
```

```
    self.is_running_sequence = True
```

```
    self.current_sequence = sequence
```

```
    self.current_position_index = 0
```

```
    # Iniciar execução da sequência
```

```
    return self._process_next_position(callback)
```

```
def _process_next_position(self, callback=None):
```

```
    """Processa a próxima posição na sequência."""
```

```
    if not self.is_running_sequence or self.current_position_index >= len(self.current_sequence.positions):
```

```
        self.is_running_sequence = False
```

```
        return None
```

```
    # Obtém a próxima posição
```

```
    position = self.current_sequence.positions[self.current_position_index]
```

```
    # Move para a posição
```

```
    self.cnc.move_to_absolute_position(position.x, position.y)
```

```
    # Verifica se chegou na posição
```

```
    while self.cnc.is_moving():
```

```
        time.sleep(0.1)
```

```
    # Captura a imagem
```

```
    image = self.camera.capture(position.camera_params)
```

```
    # Prepara dados para callback
```

```
    result = {
```

```
        "position": position,
```

```
        "image": image,
```

```
        "timestamp": time.time(),
```

```
        "sequence": self.current_sequence.name,
```

```
        "index": self.current_position_index
```

```
    }
```

```

# Avança para a próxima posição
self.current_position_index += 1

# Chama o callback se fornecido
if callback:
    callback(result)

return result

def stop_sequence(self):
    """Para a execução da sequência atual."""
    self.is_running_sequence = False

def save_positions(self, filename):
    """Salva todas as posições e sequências em um arquivo."""
    return self.position_manager.save_to_file(filename)

def load_positions(self, filename):
    """Carrega posições e sequências de um arquivo."""
    return self.position_manager.load_from_file(filename)

```

File: camera_controller.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```

class CameraController:
    """
    Classe para controlar uma câmera e capturar imagens.
    Pode ser adaptada para diferentes interfaces de câmera.
    """

    def __init__(self, camera_interface=None):
        """
        Inicializa o controlador de câmera.

        Args:
            camera_interface: Interface de câmera personalizada (opcional)
        """
        self.camera = camera_interface
        self.is_connected = False
        self.last_error = ""

    def connect(self, camera_id=0):
        """
        Conecta à câmera.

        Args:
            camera_id: ID da câmera (para OpenCV)

        Returns:
            bool: True se conectado com sucesso

```

```

"""
if self.camera:
    # Usa a interface fornecida
    try:
        self.is_connected = self.camera.connect()
        return self.is_connected
    except Exception as e:
        self.last_error = str(e)
        return False
else:
    # Implementação padrão com OpenCV
    try:
        import cv2
        self.camera = cv2.VideoCapture(camera_id)
        self.is_connected = self.camera.isOpened()
        return self.is_connected
    except Exception as e:
        self.last_error = str(e)
        return False

def disconnect(self):
    """Desconecta da câmera."""
    if not self.is_connected:
        return

    try:
        if hasattr(self.camera, 'release'):
            self.camera.release()
        elif hasattr(self.camera, 'disconnect'):
            self.camera.disconnect()

        self.is_connected = False

    except Exception as e:
        self.last_error = str(e)

def trigger_capture(self, params=None):
    """
    Aciona o trigger da câmera para captura.

    Args:
        params: Parâmetros específicos da câmera para o trigger

    Returns:
        A imagem capturada ou None em caso de erro
    """
    # Primeiro realiza o trigger
    if not self._activate_trigger(params):
        return None

    # Em seguida, captura a imagem

```

```
return self.capture(params)
```

```
def _activate_trigger(self, params=None):
```

```
    """
```

Ativa o hardware trigger da câmera, se disponível.

Args:

params: Parâmetros específicos para o trigger

Returns:

True se o trigger foi bem-sucedido, False caso contrário

```
    """
```

```
if not self.is_connected:
```

```
    self.last_error = "Câmera não conectada"
```

```
    return False
```

```
try:
```

```
    # Verifica se a interface da câmera tem um método de trigger específico
```

```
    if hasattr(self.camera, 'trigger'):
```

```
        # Interface personalizada com método de trigger
```

```
        return self.camera.trigger(params)
```

```
    # Para câmeras OpenCV padrão, criar um atraso simulando trigger por software
```

```
    # e garantir que o frame seja descartado/limpo antes da captura real
```

```
    import time
```

```
    if hasattr(self.camera, 'grab'):
```

```
        # Limpa o buffer de frames
```

```
        self.camera.grab()
```

```
        time.sleep(0.1) # Pequeno atraso para simular trigger
```

```
        return True
```

```
    return True # Se não houver trigger específico, continua com a captura normal
```

```
except Exception as e:
```

```
    self.last_error = f"Erro ao ativar trigger: {str(e)}"
```

```
    return False
```

```
def has_hardware_trigger(self):
```

```
    """
```

Verifica se a câmera possui suporte a hardware trigger.

Returns:

True se suporta trigger por hardware, False caso contrário

```
    """
```

```
if not self.is_connected:
```

```
    return False
```

```
    # Verifica se a interface da câmera tem capacidade explícita de trigger
```

```
    return hasattr(self.camera, 'trigger') or hasattr(self.camera, 'has_hardware_trigger')
```

```
def capture(self, params=None):
```

```
"""
```

Captura uma imagem da câmera.

Args:

params: Parâmetros específicos da câmera (exposição, etc.)

Returns:

A imagem capturada ou None em caso de erro

```
"""
```

```
if not self.is_connected:
```

```
    self.last_error = "Câmera não conectada"
```

```
    return None
```

```
try:
```

```
    # Para interface personalizada
```

```
    if hasattr(self.camera, 'capture'):
```

```
        image = self.camera.capture(params)
```

```
        if image is None:
```

```
            self.last_error = "A captura de imagem retornou None"
```

```
            return None
```

```
        return image
```

```
    # Para OpenCV
```

```
    import cv2
```

```
    ret, frame = self.camera.read()
```

```
    if ret and frame is not None:
```

```
        # Aplicar parâmetros se fornecidos
```

```
        if params:
```

```
            # Exemplo: ajustar brilho/contraste
```

```
            if 'brightness' in params:
```

```
                frame = cv2.convertScaleAbs(frame, alpha=params['brightness'])
```

```
    # Verifica se a imagem possui dados válidos
```

```
    if frame.size == 0 or frame.shape[0] == 0 or frame.shape[1] == 0:
```

```
        self.last_error = "Imagem capturada tem dimensões inválidas"
```

```
        return None
```

```
    return frame
```

```
else:
```

```
    self.last_error = "Falha ao capturar imagem (sem dados ou retorno negativo)"
```

```
    return None
```

```
except Exception as e:
```

```
    self.last_error = f"Erro na captura de imagem: {str(e)}"
```

```
    return None
```

```
def set_parameters(self, **kwargs):
```

```
"""
```

Define parâmetros da câmera.

Args:

```

        **kwargs: Parâmetros específicos da câmera
        """

    if not self.is_connected:
        return False

    try:
        # Para interface personalizada
        if hasattr(self.camera, 'set_parameters'):
            return self.camera.set_parameters(**kwargs)

        # Para OpenCV
        import cv2
        for key, value in kwargs.items():
            if key.lower() == 'exposure':
                self.camera.set(cv2.CAP_PROP_EXPOSURE, value)
            elif key.lower() == 'focus':
                self.camera.set(cv2.CAP_PROP_FOCUS, value)
            # Adicione outros parâmetros conforme necessário

        return True

    except Exception as e:
        self.last_error = str(e)
        return False

```

File: cnc_controller.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```

import time
import serial
import serial.tools.list_ports
import threading
from grbl_streamer import GrblStreamer # Importa a biblioteca grbl-streamer

import logging
# Se ainda não existir, defina um logger para esta classe:
logger = logging.getLogger("GRBLCNCController")
logger.setLevel(logging.DEBUG)

class EventEmitter:
    """
    Sistema de eventos para comunicação assíncrona.
    Permite registrar múltiplos callbacks para diferentes eventos.
    """

    def __init__(self):
        self.callbacks = {}
        self.default_callbacks = []

    def connect(self, callback, event_name=None):
        """

```

Conecta um callback a um evento específico ou como callback padrão.

Args:

callback: Função a ser chamada quando o evento ocorrer

event_name: Nome do evento (se None, será adicionado aos callbacks padrão)

.....

if event_name is None:

self.default_callbacks.append(callback)

else:

if event_name not in self.callbacks:

self.callbacks[event_name] = []

self.callbacks[event_name].append(callback)

Retorna uma função que pode ser usada para desconectar este callback

return lambda: self.disconnect(callback, event_name)

def disconnect(self, callback, event_name=None):

.....

Desconecta um callback específico.

Args:

callback: Callback a ser removido

event_name: Nome do evento do qual remover (se None, remove dos callbacks padrão)

.....

if event_name is None:

if callback in self.default_callbacks:

self.default_callbacks.remove(callback)

else:

if event_name in self.callbacks and callback in self.callbacks[event_name]:

self.callbacks[event_name].remove(callback)

def disconnect_all(self, event_name=None):

.....

Desconecta todos os callbacks de um evento específico ou todos os eventos.

Args:

event_name: Nome do evento para limpar (se None, limpa todos os eventos)

.....

if event_name is None:

self.callbacks.clear()

self.default_callbacks.clear()

else:

if event_name in self.callbacks:

self.callbacks[event_name].clear()

def emit(self, *args, event_name=None, **kwargs):

.....

Emite um evento, chamando todos os callbacks registrados.

Args:

*args: Argumentos posicionais para passar aos callbacks

```

    event_name: Nome do evento a ser emitido (se None, apenas callbacks padrão são chamados)
    **kwargs: Argumentos nomeados para passar aos callbacks
    """

    # Chama todos os callbacks padrão
    for callback in self.default_callbacks:
        try:
            callback(*args, **kwargs)
        except Exception as e:
            print(f"Erro em callback para evento: {e}")

```

```

    # Se um nome de evento for especificado, chama esses callbacks também
    if event_name is not None and event_name in self.callbacks:
        for callback in self.callbacks[event_name]:
            try:
                callback(*args, **kwargs)
            except Exception as e:
                print(f"Erro em callback para evento '{event_name}': {e}")

```

```

def has_listeners(self, event_name=None):
    """

```

Verifica se há callbacks registrados para um evento específico ou para qualquer evento.

Args:

event_name: Nome do evento a verificar (se None, verifica todos)

Returns:

bool: True se houver callbacks registrados, False caso contrário

```

    """

```

```

    if event_name is None:

```

```

        return bool(self.default_callbacks) or any(bool(callbacks) for callbacks in self.callbacks.values())

```

```

    else:

```

```

        return event_name in self.callbacks and bool(self.callbacks[event_name])

```

```

class GRBLCNCController:
    """

```

Classe para controlar uma máquina CNC usando GRBL.

Implementada usando a biblioteca grbl-streamer.

```

    """

```

```

    def __init__(self):

```

```

        """Inicializa o controlador CNC."""

```

```

        self.grbl = None

```

```

        self.lock = threading.Lock()

```

```

        self.is_connected = False

```

```

        self.current_position = {'x': 0, 'y': 0, 'z': 0}

```

```

        self.machine_status = "Disconnected"

```

```

        self.last_response = ""

```

```

        self.last_error = ""

```

```

    # Eventos para comunicação assíncrona

```

```

    self.response_received = EventEmitter()

```

```
self.status_update = EventEmitter()
self.position_update = EventEmitter()
self.error_message = EventEmitter()
```

```
# Thread para consulta periódica de status
```

```
self.status_thread = None
```

```
self.running = False
```

```
# Controle de jog contínuo
```

```
self.jogging = False
```

```
self.current_jog_command = None
```

```
self.steps_to_mm_factor = 1.0
```

```
def connect(self, port=None, baudrate=115200):
```

```
    """
```

```
    Conecta à máquina CNC.
```

```
    Args:
```

```
        port: Porta serial (None para autodetecção)
```

```
        baudrate: Taxa de transmissão
```

```
    Returns:
```

```
        bool: True se conectado com sucesso
```

```
    """
```

```
    try:
```

```
        # Autodetectar porta se não especificada
```

```
        if port is None:
```

```
            ports = [p.device for p in serial.tools.list_ports.comports()]
```

```
            if not ports:
```

```
                self.last_error = "Nenhuma porta serial encontrada"
```

```
                return False
```

```
            port = ports[0]
```

```
        # Inicializa e configura o GrblStreamer
```

```
        self.grbl = GrblStreamer(port=port, baud=baudrate)
```

```
        # Configura callbacks para eventos do GrblStreamer
```

```
        self._setup_callbacks()
```

```
        # Desbloqueia a máquina
```

```
        self.send_command("$X", priority=True)
```

```
        # Inicia thread de consulta de status
```

```
        self.running = True
```

```
        self.status_thread = threading.Thread(target=self._status_polling_thread, daemon=True)
```

```
        self.status_thread.start()
```

```
        self.is_connected = True
```

```
        self.machine_status = "Idle" # Estado inicial presumido
```

```
        return True
```

```
except Exception as e:
    self.last_error = str(e)
    return False
```

```
def _setup_callbacks(self):
```

```
    """
```

```
    Configura callbacks para eventos do GrblStreamer.
    Esta função deve ser adaptada com base na API real da biblioteca.
```

```
    """
```

```
    # Configuração para receber atualizações de status da máquina
    self.grbl.register_status_callback(self._on_status_update)
```

```
    # Configuração para receber respostas de comandos
    self.grbl.register_response_callback(self._on_response_received)
```

```
    # Configuração para receber notificações de erro
    self.grbl.register_error_callback(self._on_error_message)
```

```
def _status_polling_thread(self):
```

```
    """Thread para consulta periódica de status."""
```

```
    while self.running and self.is_connected:
```

```
        try:
```

```
            # Requisição de status ao GRBL a cada 200ms
```

```
            if self.grbl:
```

```
                self.grbl.get_status()
```

```
            time.sleep(0.2)
```

```
        except Exception:
```

```
            time.sleep(0.5) # Maior intervalo em caso de erro
```

```
def disconnect(self):
```

```
    """Desconecta da máquina CNC."""
```

```
    # Garante que o jog é parado antes de desconectar
```

```
    if self.jogging:
```

```
        self.stop_continuous_jog()
```

```
    self.running = False
```

```
    if self.status_thread:
```

```
        self.status_thread.join(1.0) # Espera até 1 segundo pela thread
```

```
    if self.grbl:
```

```
        try:
```

```
            self.grbl.close()
```

```
            self.grbl = None
```

```
        except Exception as e:
```

```
            self.last_error = str(e)
```

```
    self.is_connected = False
```

```
    self.machine_status = "Disconnected"
```

```
def is_moving(self):
```

```
    """Verifica se a máquina está em movimento."""
```

```
return self.machine_status == "Run" or self.machine_status == "Jog"
```

```
def wait_for_idle(self, timeout=30):
```

```
    """
```

Aguarda até que a máquina esteja ociosa.

Args:

timeout: Tempo máximo de espera em segundos

Returns:

bool: True se ficou ocioso, False se atingiu timeout

```
    """
```

```
    start_time = time.time()
```

```
    while self.is_moving():
```

```
        if time.time() - start_time > timeout:
```

```
            return False
```

```
            time.sleep(0.1)
```

```
    return True
```

```
def get_current_position(self):
```

```
    """
```

Retorna a posição atual do CNC.

Certifica-se de retornar uma cópia do dicionário para evitar alteração accidental.

```
    """
```

```
    logger.debug(f"get_current_position: Retornando posição atual: {self.current_position if hasattr(self, 'current_position')")
```

```
    if hasattr(self, 'current_position'):
```

```
        # current_position é um dicionário com chaves 'x', 'y' e 'z'
```

```
        # Retorna uma cópia para evitar alterações accidentais
```

```
        return self.current_position.copy()
```

```
    return {'x': 0, 'y': 0, 'z': 0}
```

```
def move_to_absolute_position(self, x=None, y=None, feed_rate=1000):
```

```
    """
```

Move para uma posição absoluta.

Args:

x, y: Coordenadas de destino (None para não mover esse eixo)

feed_rate: Velocidade de avanço em mm/min

```
    """
```

```
    if not self.is_connected:
```

```
        return False
```

```
    # Muda para modo absoluto
```

```
    self.send_command("G90", priority=True)
```

```
    # Constrói o comando
```

```
    command = "G1"
```

```
    if x is not None:
```

```
        command += f" X{x}"
```

```
    if y is not None:
```

```
        command += f" Y{y}"
```

```
command += f" F{feed_rate}"
```

```
# Envia o comando
```

```
self.send_command(command)
```

```
return True
```

```
def move_relative(self, x=0, y=0, feed_rate=1000):
```

```
    """
```

```
    Move em relação à posição atual.
```

```
    Args:
```

```
        x, y: Distância a mover em cada eixo
```

```
        feed_rate: Velocidade de avanço em mm/min
```

```
    """
```

```
    if not self.is_connected:
```

```
        return False
```

```
# Muda para modo relativo
```

```
self.send_command("G91", priority=True)
```

```
# Constrói o comando
```

```
command = f"G1 X{x} Y{y} F{feed_rate}"
```

```
# Envia o comando
```

```
self.send_command(command)
```

```
return True
```

```
def start_continuous_jog(self, axis, direction, feed_rate=1000):
```

```
    """
```

```
    Inicia um movimento jog contínuo em um eixo específico.
```

```
    O movimento continua até que stop_continuous_jog seja chamado.
```

```
    Args:
```

```
        axis: Eixo a mover ('X' ou 'Y')
```

```
        direction: Direção do movimento (1 ou -1)
```

```
        feed_rate: Velocidade em mm/min
```

```
    Returns:
```

```
        bool: True se o comando foi iniciado com sucesso
```

```
    """
```

```
    if not self.is_connected or self.jogging:
```

```
        return False
```

```
# Para qualquer movimento anterior e garante modo relativo
```

```
self.emergency_stop()
```

```
time.sleep(0.1) # Pequena pausa para garantir que o GRBL processou o comando de parada
```

```
self.send_command("~", priority=True) # Retoma após parada
```

```
self.send_command("G91", priority=True) # Modo relativo
```

```
# Em GRBL 1.1, podemos usar o comando de jog para movimento contínuo
```

```
# A sintaxe é: $J=G91 X[dist] Y[dist] F[feed]
```

```
distance = 1000 * direction # Distância grande para simular movimento contínuo
```

```
if axis.upper() == 'X':  
    jog_command = f"$J=G91 X{distance} F{feed_rate}"  
else: # assume Y  
    jog_command = f"$J=G91 Y{distance} F{feed_rate}"
```

```
self.jogging = True  
self.current_jog_command = jog_command
```

```
# Envia comando de jog com máxima prioridade  
success = self.send_command(jog_command, priority=True)
```

```
return success
```

```
def stop_continuous_jog(self):
```

```
    """
```

```
    Para o movimento jog contínuo atual.
```

```
    Returns:
```

```
        bool: True se o comando foi enviado com sucesso
```

```
    """
```

```
if not self.is_connected or not self.jogging:  
    return False
```

```
# O comando para cancelar jog no GRBL é o feed cancel (!)  
success = self.send_command("!", priority=True)
```

```
# Após um breve intervalo, envia comando de retomada para liberar a máquina  
time.sleep(0.1)  
self.send_command("~", priority=True)
```

```
self.jogging = False  
self.current_jog_command = None
```

```
return success
```

```
def home(self):
```

```
    """Envia a máquina para a posição home."""
```

```
if not self.is_connected:  
    return False
```

```
self.send_command("$H")  
return True
```

```
def set_zero(self):
```

```
    """Define a posição atual como zero."""
```

```
if not self.is_connected:  
    return False
```

```
self.send_command("G92 X0 Y0")
```

```
return True
```

```
def unlock(self):
```

```
    """Envia o comando de desbloqueio ($X) para o GRBL."""
```

```
    if not self.is_connected or not self.grbl:
```

```
        logger.warning("UNLOCK: CNC não conectada.")
```

```
        return False
```

```
    try:
```

```
        logger.info("UNLOCK: Enviando comando de desbloqueio ($X)")
```

```
        # A biblioteca tem um método dedicado para killalarm
```

```
        self.grbl.killalarm()
```

```
        logger.info("UNLOCK: Comando de desbloqueio enviado.")
```

```
        return True
```

```
    except Exception as e:
```

```
        logger.error(f"UNLOCK: Erro ao enviar comando de desbloqueio: {e}", exc_info=True)
```

```
        return False
```

```
def emergency_stop(self):
```

```
    """Para todos os movimentos imediatamente (Feed Hold)."""
```

```
    if not self.is_connected or not self.grbl:
```

```
        logger.warning("EMERGENCY STOP: CNC não conectada.")
```

```
        return False
```

```
    try:
```

```
        logger.info("EMERGENCY STOP: Enviando comando Feed Hold (!)")
```

```
        # CORREÇÃO: Usar send_immediately() em vez de send_realtime_command()
```

```
        self.grbl.send_immediately("!")
```

```
        self.jogging = False # Atualiza o estado de jog, pois o movimento parou
```

```
        self.machine_status = "Hold" # Estado esperado após '!' é Hold
```

```
        logger.info("EMERGENCY STOP: Comando Feed Hold (!) enviado. Estado esperado: Hold.")
```

```
        return True
```

```
    except Exception as e:
```

```
        logger.error(f"EMERGENCY STOP: Erro ao enviar comando Feed Hold (!): {e}", exc_info=True)
```

```
        return False
```

```
def send_soft_reset(self):
```

```
    """Envia um comando de Soft Reset (Ctrl+X) para o GRBL."""
```

```
    if not self.is_connected or not self.grbl:
```

```
        logger.warning("SOFT RESET: CNC não conectada.")
```

```
        return False
```

```
    try:
```

```
        # O Soft Reset no GRBL é o caractere Ctrl+X (código ASCII 0x18 ou 24 em decimal)
```

```
        logger.info("SOFT RESET: Enviando comando Soft Reset (Ctrl+X)")
```

```
        # Utiliza o método send_immediately() com o caractere adequado
```

```
        self.grbl.send_immediately("\x18")
```

```
        self.jogging = False # Assume que o reset também para o jog
```

```

        self.machine_status = "Alarm" # O estado esperado após reset é Alarme
        logger.info("SOFT RESET: Comando enviado. Estado esperado: Alarm.")
        return True
    except Exception as e:
        # Loga o erro se ocorrer
        logger.error(f"SOFT RESET: Erro ao enviar comando Soft Reset: {e}", exc_info=True)
        return False

```

```

def send_command(self, command, priority=False):

```

```

    """

```

```

    Envia um comando personalizado ao GRBL.

```

```

    Args:

```

```

        command: Comando G-code

```

```

        priority: True para enviar com prioridade

```

```

    """

```

```

    if not self.is_connected or not self.grbl:

```

```

        return False

```

```

    try:

```

```

        # Comandos de controle realtime (!, ~, ?) são tratados especialmente

```

```

        if command in ["!", "~", "?"]:

```

```

            self.grbl.send_immediately(command)

```

```

        # Comandos prioritários são enviados com prioridade

```

```

        elif priority:

```

```

            self.grbl.send_immediately(command)

```

```

        return True

```

```

    except Exception as e:

```

```

        self.last_error = str(e)

```

```

        self.error_message.emit(str(e))

```

```

        return False

```

```

def _on_response_received(self, response):

```

```

    """Callback quando uma resposta é recebida."""

```

```

    self.last_response = response

```

```

    self.response_received.emit(response)

```

```

def _on_status_update(self, status_data):

```

```

    """Callback quando o status da máquina é atualizado."""

```

```

    logger.debug(">> _on_status_update chamado com status_data: %s", status_data)

```

```

    # Extraí o estado da máquina

```

```

    state = status_data.get('state', 'Unknown')

```

```

    self.machine_status = state

```

```

    logger.debug("Estado da máquina extraído: %s", state)

```

```

    # Extraí a posição (espera-se um dicionário com chaves 'x', 'y' e 'z')

```

```

    position = status_data.get('position', None)

```

```

    if position:

```

```

        new_position = {

```

```

        'x': position.get('x', 0),
        'y': position.get('y', 0),
        'z': position.get('z', 0)
    }
    logger.debug("Posição extraída: %s", new_position)
    if new_position['x'] == 0 and new_position['y'] == 0:
        logger.warning("As coordenadas X e Y continuam 0 mesmo após movimento. status_data recebido: %s", status_data)
        self.current_position = new_position
        self.position_update.emit(self.current_position)
    else:
        logger.warning("status_data não contém a chave 'position': %s", status_data)

    status_string = f"<{state}>|MPos:{self.current_position['x']:.3f},{self.current_position['y']:.3f},{self.current_position['z']:.3f}"
    logger.debug("String de status formada: %s", status_string)
    self.status_update.emit(status_string)

def _on_error_message(self, error):
    """Callback quando ocorre um erro."""
    self.last_error = error
    self.error_message.emit(error)

```

File: gcode_manager.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

"""

Módulo para gerenciamento de G-CODE na aplicação de inspeção AOI.
 Permite criar, ler e manipular arquivos G-CODE para controle de percursos de inspeção.

"""

```

import os
import re
import time
import logging
from datetime import datetime

```

```

logger = logging.getLogger("GCodeManager")

```

```

class GCodeManager:

```

"""

Gerencia operações de G-CODE para o sistema AOI, incluindo:

- Geração de G-CODE a partir de posições de inspeção
- Leitura de arquivos G-CODE para criar sequências de inspeção
- Inclusão de comandos especiais para captura de imagens

"""

Definição de comandos especiais como comentários em G-CODE

CAPTURE_IMAGE_CMD = ";AOI_CAPTURE_IMAGE"

POSITION_NAME_CMD = ";AOI_POSITION_NAME:"

SEQUENCE_NAME_CMD = ";AOI_SEQUENCE_NAME:"

CAMERA_PARAMS_CMD = ";AOI_CAMERA_PARAMS:"

```

def __init__(self):
    """Inicializa o gerenciador de G-CODE"""
    self.feed_rate = 1000 # Taxa de avanço padrão (mm/min)
    self.safe_z_height = 5 # Altura Z para movimentos seguros
    self.use_z_axis = False # Se verdadeiro, inclui movimentos no eixo Z

def generate_gcode_from_sequence(self, sequence, include_header=True):
    """
    Gera G-CODE a partir de uma sequência de inspeção

    Args:
        sequence: Objeto InspectionSequence contendo as posições
        include_header: Se True, inclui cabeçalho com informações da sequência

    Returns:
        Uma string contendo o G-CODE completo
    """
    gcode_lines = []

    # Adiciona cabeçalho com informações
    if include_header:
        gcode_lines.extend([
            f"; AOI Inspection Sequence Generated G-CODE",
            f"; Date: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
            f"{self.SEQUENCE_NAME_CMD} {sequence.name}",
            f"; Total positions: {len(sequence.positions)}",
            f"; Feed rate: {self.feed_rate} mm/min",
            ";",
        ])

    # Configurações iniciais
    gcode_lines.extend([
        "G21 ; Set units to mm",
        "G90 ; Set absolute positioning",
        f"F{self.feed_rate} ; Set feed rate"
    ])

    # Para cada posição na sequência
    for i, position in enumerate(sequence.positions):
        # Adiciona comentário com o nome da posição
        gcode_lines.append(f"\n; Position {i+1}: {position.name}")
        gcode_lines.append(f"{self.POSITION_NAME_CMD} {position.name}")

        # Adiciona parâmetros da câmera se existirem
        if position.camera_params:
            params_str = ",".join([f"{k}:{v}" for k, v in position.camera_params.items()])
            gcode_lines.append(f"{self.CAMERA_PARAMS_CMD} {params_str}")

        # Adiciona movimento para a posição
        gcode_lines.append(f"G0 X{position.x:.3f} Y{position.y:.3f} ; Move to position")

```

```
# Adiciona comando para captura de imagem
gcode_lines.append(f"{self.CAPTURE_IMAGE_CMD} ; Capture image at this position")
```

```
# Finaliza o programa
gcode_lines.append("\nM2 ; End program")
```

```
return "\n".join(gcode_lines)
```

```
def read_gcode_file(self, filename):
```

```
    """
```

Lê um arquivo G-CODE e cria uma sequência de posições de inspeção

Args:

filename: Caminho do arquivo G-CODE

Returns:

Um dicionário contendo:

- 'sequence_name': Nome da sequência extraída do arquivo
- 'positions': Lista de dicionários com informações das posições

```
    """
```

```
if not os.path.exists(filename):
```

```
    logger.error(f"Arquivo G-CODE não encontrado: {filename}")
```

```
    return None
```

```
try:
```

```
    with open(filename, 'r') as f:
```

```
        gcode_content = f.read()
```

```
    return self.parse_gcode(gcode_content)
```

```
except Exception as e:
```

```
    logger.error(f"Erro ao ler arquivo G-CODE: {str(e)}")
```

```
    return None
```

```
def parse_gcode(self, gcode_content):
```

```
    """
```

Analisa o conteúdo G-CODE e extrai sequência e posições

Args:

gcode_content: String contendo o G-CODE a ser analisado

Returns:

Dicionário com dados da sequência e posições

```
    """
```

```
lines = gcode_content.split('\n')
```

```
sequence_name = "Sequência Importada"
```

```
positions = []
```

```
current_position = None
```

```

# Extrai nome da sequência
for line in lines:
    if self.SEQUENCE_NAME_CMD in line:
        sequence_name = line.split(self.SEQUENCE_NAME_CMD, 1)[1].strip()
        break

# Extrai as posições
for line in lines:
    # Nova posição
    if self.POSITION_NAME_CMD in line:
        # Finaliza posição anterior se existir
        if current_position:
            positions.append(current_position)

        # Inicia nova posição
        position_name = line.split(self.POSITION_NAME_CMD, 1)[1].strip()
        current_position = {
            'name': position_name,
            'x': 0.0,
            'y': 0.0,
            'camera_params': {}
        }

# Parâmetros da câmera
elif self.CAMERA_PARAMS_CMD in line and current_position:
    params_str = line.split(self.CAMERA_PARAMS_CMD, 1)[1].strip()
    params_pairs = params_str.split(',')

    for pair in params_pairs:
        if ':' in pair:
            key, value = pair.split(':', 1)
            # Tenta converter para número se possível
            try:
                if '.' in value:
                    value = float(value)
                else:
                    value = int(value)
            except ValueError:
                pass # Mantém como string

            current_position['camera_params'][key] = value

# Movimento G0/G1
elif (line.strip().startswith('G0') or line.strip().startswith('G1')) and current_position:
    # Extrai coordenadas X e Y
    x_match = re.search(r'X([-\\d.]+)', line)
    y_match = re.search(r'Y([-\\d.]+)', line)

    if x_match:
        current_position['x'] = float(x_match.group(1))
    if y_match:

```

```

        current_position['y'] = float(y_match.group(1))

# Adiciona a última posição se existir
if current_position:
    positions.append(current_position)

return {
    'sequence_name': sequence_name,
    'positions': positions
}

def save_gcode_to_file(self, sequence, filename):
    """
    Gera G-CODE a partir de uma sequência e salva em um arquivo

    Args:
        sequence: Objeto InspectionSequence contendo as posições
        filename: Caminho do arquivo onde o G-CODE será salvo

    Returns:
        Boolean indicando sucesso da operação
    """
    try:
        gcode_content = self.generate_gcode_from_sequence(sequence)

        with open(filename, 'w') as f:
            f.write(gcode_content)

        return True

    except Exception as e:
        logger.error(f"Erro ao salvar arquivo G-CODE: {str(e)}")
        return False

```

File: position_manager.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```
import time
```

```

class InspectionPosition:
    """Representa uma posição de inspeção."""

    def __init__(self, name, x, y, camera_params=None):
        self.name = name
        self.x = x
        self.y = y
        self.camera_params = camera_params or {}
        self.creation_time = time.time()

    def to_dict(self):

```

```
"""Converte para dicionário para serialização."""
```

```
return {  
    'name': self.name,  
    'x': self.x,  
    'y': self.y,  
    'camera_params': self.camera_params,  
    'creation_time': self.creation_time  
}
```

```
@classmethod
```

```
def from_dict(cls, data):
```

```
    """Cria a partir de um dicionário."""
```

```
    position = cls(data['name'], data['x'], data['y'], data['camera_params'])
```

```
    position.creation_time = data.get('creation_time', time.time())
```

```
    return position
```

```
class InspectionSequence:
```

```
    """Representa uma sequência de posições de inspeção."""
```

```
def __init__(self, name):
```

```
    self.name = name
```

```
    self.positions = []
```

```
    self.creation_time = time.time()
```

```
    self.last_run_time = None
```

```
def add_position(self, position):
```

```
    """Adiciona uma posição à sequência."""
```

```
    self.positions.append(position)
```

```
def remove_position(self, index):
```

```
    """Remove uma posição pelo índice."""
```

```
    if 0 <= index < len(self.positions):
```

```
        del self.positions[index]
```

```
        return True
```

```
    return False
```

```
def move_position(self, old_index, new_index):
```

```
    """Move uma posição na sequência."""
```

```
    if 0 <= old_index < len(self.positions) and 0 <= new_index < len(self.positions):
```

```
        position = self.positions.pop(old_index)
```

```
        self.positions.insert(new_index, position)
```

```
        return True
```

```
    return False
```

```
def to_dict(self):
```

```
    """Converte para dicionário para serialização."""
```

```
    return {
```

```
        'name': self.name,
```

```
        'positions': [p.to_dict() for p in self.positions],
```

```
        'creation_time': self.creation_time,
```

```
        'last_run_time': self.last_run_time
    }
```

```
@classmethod
```

```
def from_dict(cls, data, positions_dict=None):
```

```
    """Cria a partir de um dicionário."""
```

```
    sequence = cls(data['name'])
```

```
    sequence.creation_time = data.get('creation_time', time.time())
```

```
    sequence.last_run_time = data.get('last_run_time')
```

```
    # Reconstruir posições
```

```
    for pos_data in data['positions']:
```

```
        if positions_dict and pos_data['name'] in positions_dict:
```

```
            # Usa posição existente
```

```
            sequence.positions.append(positions_dict[pos_data['name']])
```

```
        else:
```

```
            # Cria nova posição
```

```
            position = InspectionPosition.from_dict(pos_data)
```

```
            sequence.positions.append(position)
```

```
    return sequence
```

```
class InspectionPositionManager:
```

```
    """Gerencia posições e sequências de inspeção."""
```

```
    def __init__(self):
```

```
        self.positions = {} # name -> position
```

```
        self.sequences = {} # name -> sequence
```

```
    def add_position(self, position):
```

```
        """Adiciona uma posição."""
```

```
        self.positions[position.name] = position
```

```
    def remove_position(self, position_name):
```

```
        """Remove uma posição pelo nome."""
```

```
        if position_name in self.positions:
```

```
            del self.positions[position_name]
```

```
        # Atualiza sequências que usam esta posição
```

```
        for sequence in self.sequences.values():
```

```
            sequence.positions = [p for p in sequence.positions if p.name != position_name]
```

```
        return True
```

```
        return False
```

```
    def get_position(self, position_name):
```

```
        """Obtém uma posição pelo nome."""
```

```
        return self.positions.get(position_name)
```

```
    def add_sequence(self, sequence):
```

```

"""Adiciona uma sequência."""
self.sequences[sequence.name] = sequence

def remove_sequence(self, sequence_name):
    """Remove uma sequência pelo nome."""
    if sequence_name in self.sequences:
        del self.sequences[sequence_name]
        return True
    return False

def get_sequence(self, sequence_name):
    """Obtém uma sequência pelo nome."""
    return self.sequences.get(sequence_name)

def save_to_file(self, filename):
    """Salva todas as posições e sequências em um arquivo JSON."""
    data = {
        'positions': {name: pos.to_dict() for name, pos in self.positions.items()},
        'sequences': {name: seq.to_dict() for name, seq in self.sequences.items()}
    }

    try:
        with open(filename, 'w') as f:
            import json
            json.dump(data, f, indent=2)
        return True
    except Exception as e:
        print(f"Erro ao salvar dados: {e}")
        return False

def load_from_file(self, filename):
    """Carrega posições e sequências de um arquivo JSON."""
    try:
        with open(filename, 'r') as f:
            import json
            data = json.load(f)

        # Limpa dados existentes
        self.positions.clear()
        self.sequences.clear()

        # Carrega posições
        for name, pos_data in data.get('positions', {}).items():
            position = InspectionPosition.from_dict(pos_data)
            self.positions[name] = position

        # Carrega sequências
        for name, seq_data in data.get('sequences', {}).items():
            sequence = InspectionSequence.from_dict(seq_data, self.positions)
            self.sequences[name] = sequence

```

```
    return True

except Exception as e:
    print(f"Erro ao carregar dados: {e}")
    return False
```

File: __init__.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```
# Exporta as classes principais para facilitar importação
from .aoi_controller import CNCAOIController
from .cnc_controller import GRBLCNCCController
from .camera_controller import CameraController
from .position_manager import InspectionPosition, InspectionSequence

__version__ = '0.1.0'
```

File: config_manager.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib\utils

File: __init__.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib\utils

File: consumo_lib.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi

```
import sys
import cv2
import time
import numpy as np

from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
                             QPushButton, QLabel, QGroupBox, QGridLayout, QLineEdit,
                             QComboBox, QListWidget, QListWidgetItem, QFileDialog, QMessageBox, QTabWidget,
                             QSplitter, QFrame, QTableWidget, QTableWidgetItem, QHeaderView, QDialog)

from PyQt6.QtCore import Qt, QThread, pyqtSignal, QTimer
from PyQt6.QtGui import QPixmap, QImage, QFont, QAction

from aoi_lib import CNCAOIController, InspectionPosition
from grbl_streamer import GrblStreamer

import logging
logger = logging.getLogger("consumo_lib")
logger.setLevel(logging.DEBUG)
# Se necessário, adicione um handler:
```

```
if not logger.handlers:
```

```
    ch = logging.StreamHandler()
```

```
    ch.setLevel(logging.DEBUG)
```

```
    formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(message)s")
```

```
    ch.setFormatter(formatter)
```

```
    logger.addHandler(ch)
```

```
class ImageViewerWidget(QWidget):
```

```
    """Widget para exibir imagens capturadas pela câmera"""
```

```
    def __init__(self, parent=None):
```

```
        super().__init__(parent)
```

```
        self.layout = QVBoxLayout(self)
```

```
        # Label para exibir a imagem
```

```
        self.image_label = QLabel()
```

```
        self.image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
```

```
        self.image_label.setText("Nenhuma imagem capturada")
```

```
        self.image_label.setStyleSheet("border: 1px solid gray; background-color: #f0f0f0;")
```

```
        self.image_label.setMinimumSize(400, 300)
```

```
        # Informações da imagem
```

```
        self.info_label = QLabel("Informações da imagem:")
```

```
        self.layout.addWidget(self.image_label)
```

```
        self.layout.addWidget(self.info_label)
```

```
    def display_image(self, image, info_text=None):
```

```
        """Mostra uma imagem no widget"""
```

```
        if image is None:
```

```
            self.image_label.setText("Imagem inválida")
```

```
            return
```

```
        # Converte imagem OpenCV para QPixmap
```

```
        h, w, c = image.shape
```

```
        bytes_per_line = 3 * w
```

```
        q_img = QImage(image.data, w, h, bytes_per_line, QImage.Format.Format_RGB888).rgbSwapped()
```

```
        pixmap = QPixmap.fromImage(q_img)
```

```
        # Redimensiona se for muito grande
```

```
        if pixmap.width() > 800 or pixmap.height() > 600:
```

```
            pixmap = pixmap.scaled(800, 600, Qt.AspectRatioMode.KeepAspectRatio)
```

```
        self.image_label.setPixmap(pixmap)
```

```
        # Atualiza informações
```

```
        if info_text:
```

```
            self.info_label.setText(info_text)
```

```
        else:
```

```
            self.info_label.setText(f"Imagem: {w}x{h}px")
```

```
class PositionListWidget(QWidget):
```

```

"""Widget para gerenciar lista de posições"""
position_selected = pyqtSignal(object) # Emite a posição selecionada

def __init__(self, parent=None):
    super().__init__(parent)
    self.layout = QVBoxLayout(self)

    # Título
    title_label = QLabel("Posições de Inspeção")
    title_font = QFont()
    title_font.setBold(True)
    title_label.setFont(title_font)

    # Lista de posições
    self.positions_list = QListWidget()
    self.positions_list.currentItemChanged.connect(self.on_position_selected)

    # Botões
    buttons_layout = QHBoxLayout()
    self.add_position_btn = QPushButton("Adicionar Posição Atual")
    self.remove_position_btn = QPushButton("Remover")
    buttons_layout.addWidget(self.add_position_btn)
    buttons_layout.addWidget(self.remove_position_btn)

    self.layout.addWidget(title_label)
    self.layout.addWidget(self.positions_list)
    self.layout.addLayout(buttons_layout)

    # Dicionário para mapear itens da lista para objetos de posição
    self.positions = {}

def add_position(self, position):
    """Adiciona uma posição à lista"""
    item_text = f"{{position.name}} ({{position.x:.2f}}, {{position.y:.2f}})"
    item = QListWidgetItem(item_text)
    self.positions_list.addItem(item)
    self.positions[id(item)] = position

def remove_selected_position(self):
    """Remove a posição selecionada"""
    current_item = self.positions_list.currentItem()
    if current_item:
        position = self.positions.pop(id(current_item))
        row = self.positions_list.row(current_item)
        self.positions_list.takeItem(row)
        return position
    return None

def clear_positions(self):
    """Limpa todas as posições"""
    self.positions_list.clear()

```

```

self.positions = {}

def get_all_positions(self):
    """Retorna todas as posições"""
    return list(self.positions.values())

def on_position_selected(self, current, previous):
    """Manipula evento de seleção de posição"""
    if current and current in self.positions:
        self.position_selected.emit(self.positions[current])

class SequenceControlWidget(QWidget):
    """Widget para controlar a execução da sequência"""
    def __init__(self, parent=None):
        super().__init__(parent)
        self.layout = QVBoxLayout(self)

        # Grupo de sequência
        sequence_group = QGroupBox("Controle de Sequência")
        sequence_layout = QGridLayout()

        # Nome da sequência
        sequence_layout.addWidget(QLabel("Nome:"), 0, 0)
        self.sequence_name = QLineEdit("Sequência PCB")
        sequence_layout.addWidget(self.sequence_name, 0, 1)

        # Botões de controle
        self.create_sequence_btn = QPushButton("Criar Sequência")
        self.run_sequence_btn = QPushButton("Executar Sequência")
        self.stop_sequence_btn = QPushButton("Parar")
        self.stop_sequence_btn.setEnabled(False)

        sequence_layout.addWidget(self.create_sequence_btn, 1, 0)
        sequence_layout.addWidget(self.run_sequence_btn, 1, 1)
        sequence_layout.addWidget(self.stop_sequence_btn, 2, 0, 1, 2)

        # Status
        sequence_layout.addWidget(QLabel("Status:"), 3, 0)
        self.sequence_status = QLabel("Pronto")
        sequence_layout.addWidget(self.sequence_status, 3, 1)

        sequence_group.setLayout(sequence_layout)

        # Grupo de arquivo
        file_group = QGroupBox("Salvar/Carregar")
        file_layout = QVBoxLayout()
        self.save_btn = QPushButton("Salvar Programa (JSON)")
        self.load_btn = QPushButton("Carregar Programa (JSON)")
        self.save_gcode_btn = QPushButton("Exportar para G-CODE")
        self.load_gcode_btn = QPushButton("Importar de G-CODE")
        file_layout.addWidget(self.save_btn)

```

```
file_layout.addWidget(self.load_btn)
file_layout.addWidget(self.save_gcode_btn)
file_layout.addWidget(self.load_gcode_btn)
```

```
file_group.setLayout(file_layout)
```

```
self.layout.addWidget(sequence_group)
self.layout.addWidget(file_group)
self.layout.addStretch()
```

```
class PositionRegistryWidget(QWidget):
```

```
    """Widget for showing registered positions"""
```

```
    def __init__(self, controller, parent=None):
```

```
        super().__init__(parent)
```

```
        self.controller = controller
```

```
        self.positions = [] # List of registered positions
```

```
        self.setup_ui()
```

```
    def setup_ui(self):
```

```
        layout = QVBoxLayout(self)
```

```
        # Title
```

```
        title_label = QLabel("Registered Positions")
```

```
        title_font = QFont()
```

```
        title_font.setBold(True)
```

```
        title_label.setFont(title_font)
```

```
        layout.addWidget(title_label)
```

```
        # Registered positions table
```

```
        self.positions_table = QTableWidgetItem(0, 3)
```

```
        self.positions_table.setHorizontalHeaderLabels(["Name", "X (mm)", "Y (mm)"])
```

```
        self.positions_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
```

```
        layout.addWidget(self.positions_table)
```

```
        # Buttons
```

```
        buttons_layout = QHBoxLayout()
```

```
        self.delete_button = QPushButton("Delete Selected")
```

```
        self.delete_button.clicked.connect(self.delete_position)
```

```
        self.create_sequence_btn = QPushButton("Create Sequence")
```

```
        buttons_layout.addWidget(self.delete_button)
```

```
        buttons_layout.addWidget(self.create_sequence_btn)
```

```
        layout.addLayout(buttons_layout)
```

```
    def add_position(self, name, x, y, image=None):
```

```
        """Add a position to the registry"""
```

```
        # Create position object
```

```
        position = {
```

```
    'name': name,  
    'x': x,  
    'y': y,  
    'image': image,  
}
```

```
# Add to internal list  
self.positions.append(position)
```

```
# Add to table  
row = self.positions_table.rowCount()  
self.positions_table.insertRow(row)  
self.positions_table.setItem(row, 0, QTableWidgetItem(name))  
self.positions_table.setItem(row, 1, QTableWidgetItem(f'{x:.3f}'))  
self.positions_table.setItem(row, 2, QTableWidgetItem(f'{y:.3f}'))
```

```
def delete_position(self):  
    """Delete selected position"""  
    selected_rows = self.positions_table.selectedItems()  
    if not selected_rows:  
        return  
  
    row = selected_rows[0].row()  
    if row >= 0 and row < len(self.positions):  
        del self.positions[row]  
        self.positions_table.removeRow(row)
```

```
def clear_positions(self):  
    """Clear all positions"""  
    self.positions.clear()  
    while self.positions_table.rowCount() > 0:  
        self.positions_table.removeRow(0)
```

```
class CameraPreviewWidget(QWidget):  
    """Widget for displaying camera preview and capturing images"""  
    image_captured = pyqtSignal(object, str) # Emits the captured image and position name
```

```
def __init__(self, controller, parent=None):  
    super().__init__(parent)  
    self.controller = controller  
    self.current_image = None  
    self.preview_timer = QTimer(self)  
    self.preview_timer.timeout.connect(self.update_preview)  
    self.setup_ui()
```

```
def setup_ui(self):  
    layout = QVBoxLayout(self)  
  
    # Preview area - Ajustar para usar mais espaço  
    self.image_label = QLabel()  
    self.image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
```

```
self.image_label.setText("Camera Preview")
self.image_label.setStyleSheet("border: 1px solid gray; background-color: #f0f0f0;")
self.image_label.setMinimumSize(600, 450) # Aumentar tamanho mínimo
```

```
# Camera controls
```

```
controls_layout = QHBoxLayout()
self.start_preview_btn = QPushButton("Start Preview")
self.start_preview_btn.clicked.connect(self.start_preview)
self.stop_preview_btn = QPushButton("Stop Preview")
self.stop_preview_btn.clicked.connect(self.stop_preview)
self.stop_preview_btn.setEnabled(False)
controls_layout.addWidget(self.start_preview_btn)
controls_layout.addWidget(self.stop_preview_btn)
```

```
# Position capture layout
```

```
capture_layout = QHBoxLayout()
self.position_name = QLineEdit()
self.position_name.setPlaceholderText("Position Name")
self.capture_button = QPushButton("Capture Image & Register Position")
self.capture_button.clicked.connect(self.capture_image)
capture_layout.addWidget(self.position_name)
capture_layout.addWidget(self.capture_button)
```

```
# Adicionar elementos ao layout principal
```

```
layout.addWidget(self.image_label, 1) # Proporção 1 para permitir expansão
layout.addLayout(controls_layout)
layout.addLayout(capture_layout)
```

```
def start_preview(self):
```

```
    """Start camera preview"""
```

```
    if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
```

```
        QMessageBox.warning(self, "Error", "Camera not connected")
```

```
        return
```

```
    self.preview_timer.start(100) # Update every 100ms
```

```
    self.start_preview_btn.setEnabled(False)
```

```
    self.stop_preview_btn.setEnabled(True)
```

```
def stop_preview(self):
```

```
    """Stop camera preview"""
```

```
    self.preview_timer.stop()
```

```
    self.start_preview_btn.setEnabled(True)
```

```
    self.stop_preview_btn.setEnabled(False)
```

```
def update_preview(self):
```

```
    """Update the camera preview"""
```

```
    try:
```

```
        image = self.controller.camera.capture()
```

```
        if image is not None:
```

```
            self.display_image(image)
```

```
            self.current_image = image
```

```
    except Exception as e:
```

```
print(f"Error updating preview: {e}")
self.stop_preview()
```

```
def capture_image(self):
    """Capture an image and emit signal with position name"""
    if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
        QMessageBox.warning(self, "Error", "Camera not connected")
        return
    position_name = self.position_name.text()
    if not position_name:
        QMessageBox.warning(self, "Error", "Please enter a position name")
        return
    try:
        image = self.controller.camera.capture()
        if image is not None:
            self.display_image(image)
            self.current_image = image
            self.image_captured.emit(image, position_name)
            self.position_name.clear()
        else:
            QMessageBox.warning(self, "Error", "Failed to capture image")
    except Exception as e:
        QMessageBox.warning(self, "Error", f"Capture error: {e}")
```

```
def display_image(self, image):
    """Display an image in the preview area with a crosshair in the center"""
    if image is None:
        return
```

```
# Criar uma cópia da imagem para não modificar a original
display_img = image.copy()
```

```
# Desenhar a cruz vermelha no centro
h, w, _ = display_img.shape
center_x, center_y = w // 2, h // 2
```

```
# Parâmetros da cruz
color = (0, 0, 255) # Vermelho em BGR
thickness = 2
length = min(w, h) // 20 # 5% do tamanho da dimensão menor
```

```
# Desenhar a cruz
# Linha horizontal
cv2.line(display_img,
         (center_x - length, center_y),
         (center_x + length, center_y),
         color, thickness)
# Linha vertical
cv2.line(display_img,
         (center_x, center_y - length),
         (center_x, center_y + length),
```

```
color, thickness)
```

```
# Converter a imagem OpenCV para QPixmap
```

```
h, w, c = display_img.shape
```

```
bytes_per_line = 3 * w
```

```
q_img = QImage(display_img.data, w, h, bytes_per_line, QImage.Format.Format_RGB888).rgbSwapped()
```

```
pixmap = QPixmap.fromImage(q_img)
```

```
# Obter tamanho do widget de exibição
```

```
label_width = self.image_label.width()
```

```
label_height = self.image_label.height()
```

```
# Redimensionar a imagem para caber no espaço disponível mantendo proporções
```

```
pixmap = pixmap.scaled(label_width, label_height,
```

```
Qt.AspectRatioMode.KeepAspectRatio,
```

```
Qt.TransformationMode.SmoothTransformation)
```

```
self.image_label.setPixmap(pixmap)
```

```
def resizeEvent(self, event):
```

```
    """Override do evento de redimensionamento para ajustar a imagem quando o widget for redimensionado"""
```

```
    super().resizeEvent(event)
```

```
    if self.current_image is not None:
```

```
        self.display_image(self.current_image)
```

```
class MovementControlWidget(QWidget):
```

```
    """Widget for controlling CNC movement (jog)"""
```

```
    def __init__(self, controller, parent=None):
```

```
        super().__init__(parent)
```

```
        self.controller = controller
```

```
        self.setup_ui()
```

```
    def setup_ui(self):
```

```
        layout = QVBoxLayout(self)
```

```
        # Group box for movement controls
```

```
        movement_group = QGroupBox("Movement Controls")
```

```
        movement_layout = QGridLayout()
```

```
        # Directional control buttons
```

```
        self.up_button = QPushButton("↑")
```

```
        self.down_button = QPushButton("↓")
```

```
        self.left_button = QPushButton("←")
```

```
        self.right_button = QPushButton("→")
```

```
        # Style the buttons
```

```
        for btn in [self.up_button, self.down_button, self.left_button, self.right_button]:
```

```
            btn.setMinimumSize(50, 50)
```

```
            font = QFont()
```

```
            font.setBold(True)
```

```
            font.setPointSize(16)
```

```
btn.setFont(font)
```

```
# Connect press/release events for continuous movement
```

```
self.up_button.pressed.connect(lambda: self.start_movement("Y", 1))
```

```
self.up_button.released.connect(self.stop_movement)
```

```
self.down_button.pressed.connect(lambda: self.start_movement("Y", -1))
```

```
self.down_button.released.connect(self.stop_movement)
```

```
self.left_button.pressed.connect(lambda: self.start_movement("X", -1))
```

```
self.left_button.released.connect(self.stop_movement)
```

```
self.right_button.pressed.connect(lambda: self.start_movement("X", 1))
```

```
self.right_button.released.connect(self.stop_movement)
```

```
# Add buttons to grid
```

```
movement_layout.addWidget(self.up_button, 0, 1)
```

```
movement_layout.addWidget(self.left_button, 1, 0)
```

```
movement_layout.addWidget(self.right_button, 1, 2)
```

```
movement_layout.addWidget(self.down_button, 2, 1)
```

```
# Botão de Emergency Stop / Reset
```

```
self.emergency_stop_button = QPushButton("STOP")
```

```
self.emergency_stop_button.setCheckable(True) # Torna o botão toggle
```

```
self.emergency_stop_button.setMinimumSize(100, 40)
```

```
font_stop = QFont()
```

```
font_stop.setBold(True)
```

```
self.emergency_stop_button.setFont(font_stop)
```

```
# Estilo inicial (vermelho)
```

```
self.emergency_stop_button.setStyleSheet("background-color: red; color: white;")
```

```
self.emergency_stop_button.toggled.connect(self.on_emergency_stop_toggle) # Conecta ao handler
```

```
# Adiciona o botão ao layout, centralizado abaixo dos direcionais
```

```
movement_layout.addWidget(self.emergency_stop_button, 1, 1, Qt.AlignmentFlag.AlignCenter) # Coloca no centro (lin
```

```
# Step size and feed rate controls
```

```
step_layout = QHBoxLayout()
```

```
step_layout.addWidget(QLabel("Step Size:"))
```

```
self.step_size = QLineEdit("1.0")
```

```
step_layout.addWidget(self.step_size)
```

```
step_layout.addWidget(QLabel("mm"))
```

```
feed_layout = QHBoxLayout()
```

```
feed_layout.addWidget(QLabel("Feed Rate:"))
```

```
self.feed_rate = QLineEdit("1000")
```

```
feed_layout.addWidget(self.feed_rate)
```

```
feed_layout.addWidget(QLabel("mm/min"))
```

```
# Add step and feed rate controls
```

```
movement_layout.addLayout(step_layout, 3, 0, 1, 3)
```

```
movement_layout.addLayout(feed_layout, 4, 0, 1, 3)
```

```
# Movement mode (G90/G91)
```

```
mode_layout = QHBoxLayout()
```

```
self.mode_absolute = QPushButton("Passo a Passo (G90)")
self.mode_absolute.setCheckable(True)
self.mode_absolute.clicked.connect(lambda: self.set_motion_mode("G90"))
```

```
self.mode_relative = QPushButton("Contínuo (G91)")
self.mode_relative.setCheckable(True)
self.mode_relative.setChecked(True) # Default to relative mode
self.mode_relative.clicked.connect(lambda: self.set_motion_mode("G91"))
```

```
mode_layout.addWidget(self.mode_absolute)
mode_layout.addWidget(self.mode_relative)
movement_layout.addLayout(mode_layout, 5, 0, 1, 3)
```

```
movement_group.setLayout(movement_layout)
layout.addWidget(movement_group)
```

```
def start_movement(self, axis, direction):
```

```
    """Inicia movimento no eixo e direção especificados"""
```

```
    logger.debug(f"MOVIMENTO: Iniciando movimento: eixo={axis}, direção={direction}")
```

```
    if not hasattr(self.controller.cnc, 'grbl') or not self.controller.cnc.is_connected:
```

```
        logger.warning("MOVIMENTO: CNC não conectado")
```

```
        QMessageBox.warning(self, "Error", "CNC not connected")
```

```
        return
```

```
    try:
```

```
        # Captura posição antes do movimento para verificação posterior (mantido para log, mas não usado para forçar '?')
```

```
        original_position = self.controller.cnc.get_current_position()
```

```
        logger.info(f"MOVIMENTO: Posição antes do movimento: {original_position}")
```

```
        step_size = float(self.step_size.text())
```

```
        feed_rate = float(self.feed_rate.text())
```

```
        # Verifica o modo atual
```

```
        is_absolute_mode = self.mode_absolute.isChecked()
```

```
        logger.debug(f"MOVIMENTO: Modo atual: {'Absoluto (G90)' if is_absolute_mode else 'Relativo (G91)'}")
```

```
    if is_absolute_mode:
```

```
        # MODO ABSOLUTO (G90) - Lógica Simplificada
```

```
        # 1. Calcula a posição absoluta de destino
```

```
        current_pos = self.controller.cnc.get_current_position()
```

```
        step = step_size * direction
```

```
        target_x = current_pos['x']
```

```
        target_y = current_pos['y']
```

```
        command = ""
```

```
        if axis.upper() == 'X':
```

```
            target_x += step
```

```
        # 2. Cria um único comando G90 G1
```

```
        command = f"G90 G1 X{target_x:.4f} F{feed_rate}"
```

```
    else: # Y
```

```

target_y += step
# 2. Cria um único comando G90 G1
command = f"G90 G1 Y{target_y:.4f} F{feed_rate}"

```

```

# 3. Envia o comando único
if command:
    logger.debug(f"MOVIMENTO: Enviando comando absoluto direto: {command}")
    self.controller.cnc.grbl.send_immediately(command)
else:
    logger.warning("MOVIMENTO: Não foi possível gerar comando absoluto.")

```

```

else:
    distance = 100 * direction # Distância grande para movimento contínuo

```

```

if axis.upper() == 'X':
    command = f"$J=G91 X{distance} F{feed_rate}"
else: # Y
    command = f"$J=G91 Y{distance} F{feed_rate}"

```

```

logger.debug(f"MOVIMENTO: Enviando comando contínuo: {command}")
self.controller.cnc.grbl.send_immediately(command)

```

```

except Exception as e:
    logger.error(f"MOVIMENTO: Erro ao iniciar movimento: {str(e)}")
    QMessageBox.warning(self, "Error", f"Error starting movement: {str(e)}")

```

A função verify_position_changed pode ser mantida no código, mas não será mais chamada
a partir de start_movement, evitando o envio prematuro de '?'.

```

def verify_position_changed(self, axis, direction, original_position):
    """Verifica se a posição realmente mudou após comando de movimento"""

```

```

try:
    current_position = self.controller.cnc.get_current_position()
    expected_axis = 'x' if axis.upper() == 'X' else 'y'

```

```

    logger.info(f"VERIFICAÇÃO: Movimento {axis}{'+' if direction > 0 else '-'}: "
               f"Original={original_position[expected_axis]}, "
               f"Atual={current_position[expected_axis]}")

```

```

    if original_position[expected_axis] == current_position[expected_axis]:
        logger.warning(f"VERIFICAÇÃO: Posição {expected_axis} NÃO MUDOU após comando!")

```

```

    # REMOVIDO O ENVIO FORÇADO DE STATUS:
    # if hasattr(self.controller.cnc, 'grbl') and self.controller.cnc.grbl:
    #     logger.debug("VERIFICAÇÃO: Enviando comando de status ? para atualizar posição")
    #     self.controller.cnc.grbl.send_immediately("?")

```

```

except Exception as e:
    logger.error(f"VERIFICAÇÃO: Erro ao verificar posição: {e}")

```

```

def verify_position_changed(self, axis, direction, original_position):
    """Verifica se a posição realmente mudou após comando de movimento"""
    try:

```

```
current_position = self.controller.cnc.get_current_position()
expected_axis = 'x' if axis.upper() == 'X' else 'y'
```

```
logger.info(f"VERIFICAÇÃO: Movimento {axis}{'+' if direction > 0 else '-'}: "
           f"Original={original_position[expected_axis]}, "
           f"Atual={current_position[expected_axis]}")
```

```
if original_position[expected_axis] == current_position[expected_axis]:
    logger.warning(f"VERIFICAÇÃO: Posição {expected_axis} NÃO MUDOU após comando!")
```

```
# Forçar atualização de status para tentar obter a posição atual
if hasattr(self.controller.cnc, 'grbl') and self.controller.cnc.grbl:
    logger.debug("VERIFICAÇÃO: Enviando comando de status ? para atualizar posição")
    self.controller.cnc.grbl.send_immediately("?")
```

```
except Exception as e:
```

```
    logger.error(f"VERIFICAÇÃO: Erro ao verificar posição: {e}")
```

```
def stop_movement(self):
```

```
    """Para o movimento"""
```

```
    logger.debug("MOVIMENTO: Parando movimento")
```

```
    if not hasattr(self.controller.cnc, 'grbl') or not self.controller.cnc.is_connected:
        return
```

```
    try:
```

```
        # Verifica o modo atual
```

```
        is_absolute_mode = self.mode_absolute.isChecked()
```

```
    if is_absolute_mode:
```

```
        # No modo absoluto (passo a passo), não precisamos parar o movimento
```

```
        # porque ele já deve ter sido concluído quando o botão é liberado (com a nova lógica)
```

```
        logger.debug("MOVIMENTO: Modo absoluto (passo a passo), ignorando comando de parada explícito")
```

```
        return
```

```
    else:
```

```
        # Apenas no modo contínuo (G91 com $J=) enviamos os comandos de parada
```

```
        # O comando correto para cancelar Jog é 0x85 (Jog Cancel)
```

```
        # Se a biblioteca não abstrai isso, precisamos enviar o byte.
```

```
        # Assumindo que send_immediately pode enviar bytes:
```

```
        logger.debug("MOVIMENTO: Enviando comando de cancelamento de Jog (0x85)")
```

```
        # Tentar enviar o byte de cancelamento de Jog
```

```
        try:
```

```
            # A biblioteca grbl_streamer pode não ter um método direto para bytes.
```

```
            # Se `send_immediately` falhar com bytes, pode ser necessário
```

```
            # investigar a API da biblioteca ou usar `!` como paliativo,
```

```
            # sabendo que pode não ser o ideal para $J=
```

```
            # self.controller.cnc.grbl.serial_port.write(b'\x85') # Acesso direto (não recomendado se houver abstração)
```

```
            self.controller.cnc.grbl.send_immediately("!") # Usando Feed Hold como alternativa
```

```
            logger.debug("MOVIMENTO: Comando '!' (Feed Hold) enviado como alternativa para parar Jog")
```

```
        # Agendando retomada e atualização (mantido, mas sem _force_position_update)
```

```
        logger.debug("MOVIMENTO: Agendando comando de retomada (~) após pausa")
```

```
        QTimer.singleShot(100, lambda: self._execute_resume_and_update())
```

```
except AttributeError as ae:
```

```
    logger.error(f"MOVIMENTO: Erro ao tentar enviar comando de parada de Jog. A biblioteca pode não suportar  
    # Fallback para Feed Hold se o envio direto falhar  
    self.controller.cnc.grbl.send_immediately("!")  
    logger.warning("MOVIMENTO: Usando '!' (Feed Hold) como fallback para parar Jog.")  
    QTimer.singleShot(100, lambda: self._execute_resume_and_update())
```

```
except Exception as e:
```

```
    logger.error(f"MOVIMENTO: Erro ao parar movimento: {e}")
```

```
def _execute_resume_and_update(self):
```

```
    """Executa o resumo após parada e força atualização de posição em sequência"""
```

```
    try:
```

```
        # Primeiro envia o comando para retomar após hold  
        self.controller.cnc.grbl.send_immediately("~")  
        logger.debug("MOVIMENTO: Enviado comando de retomada (~")  
  
        # --- INÍCIO DA MODIFICAÇÃO ---  
        # REMOVIDO: Chamada para _force_position_update. A atualização agora  
        # dependerá do polling regular iniciado em connect_cnc.  
        # QTimer.singleShot(150, lambda: self._force_position_update(1))  
        logger.debug("MOVIMENTO: Atualização de posição dependerá do polling regular.")  
        # --- FIM DA MODIFICAÇÃO ---
```

```
except Exception as e:
```

```
    logger.error(f"MOVIMENTO: Erro ao executar sequência de retomada: {e}")
```

```
# A função _force_position_update pode ser mantida, mas não será mais chamada
```

```
# a partir de _execute_resume_and_update.
```

```
def _force_position_update(self, attempt=1):
```

```
    """Força múltiplas atualizações de posição para garantir precisão
```

```
    Args:
```

```
        attempt: Número da tentativa atual (para limitar tentativas)
```

```
    """
```

```
    if not hasattr(self.controller.cnc, 'grbl') or not self.controller.cnc.is_connected:
```

```
        return
```

```
    try:
```

```
        # Envia comando de status para obter posição atualizada  
        # --- INÍCIO DA MODIFICAÇÃO ---  
        # logger.debug(f"MOVIMENTO: Forçando atualização de posição - tentativa {attempt}") # REMOVIDO/COMENTADO  
        # --- FIM DA MODIFICAÇÃO ---  
        self.controller.cnc.grbl.send_immediately("?")
```

```
    # Se ainda estamos dentro do limite de tentativas, agenda outra verificação
```

```
    if attempt < 3:
```

```
        # Aumento progressivo do tempo entre tentativas (150ms, 200ms, 250ms)  
        delay = 150 + (attempt * 50)
```

```
# A chamada recursiva ainda existe, mas o log dentro dela foi removido
QTimer.singleShot(delay, lambda: self._force_position_update(attempt + 1))
```

```
except Exception as e:
```

```
# Manter o log de erro
```

```
logger.error(f"MOVIMENTO: Erro ao forçar atualização de posição: {e}")
```

```
def set_motion_mode(self, mode):
```

```
    """Set the motion mode (G90/G91)"""
```

```
    if not self.controller.cnc.is_connected:
```

```
        # Se não estiver conectado, apenas atualiza a UI
```

```
        if mode == "G90":
```

```
            self.mode_absolute.setChecked(True)
```

```
            self.mode_relative.setChecked(False)
```

```
        else:
```

```
            self.mode_absolute.setChecked(False)
```

```
            self.mode_relative.setChecked(True)
```

```
        logger.warning(f"MOVIMENTO: CNC não conectada, modo {mode} definido apenas na UI.")
```

```
        return
```

```
# Atualiza a UI
```

```
if mode == "G90":
```

```
    self.mode_absolute.setChecked(True)
```

```
    self.mode_relative.setChecked(False)
```

```
else: # G91
```

```
    self.mode_absolute.setChecked(False)
```

```
    self.mode_relative.setChecked(True)
```

```
# Envia o comando para o GRBL
```

```
try:
```

```
    # --- INÍCIO DA MODIFICAÇÃO ---
```

```
    # Armazena o modo internamente para referência futura, se necessário
```

```
    # self.controller.cnc.current_motion_mode = mode # Supondo que exista essa variável no controller
```

```
    # --- FIM DA MODIFICAÇÃO ---
```

```
    logger.debug(f"MOVIMENTO: Definindo modo de movimento para {mode}")
```

```
    self.controller.cnc.grbl.send_immediately(mode)
```

```
except Exception as e:
```

```
    logger.error(f"MOVIMENTO: Erro ao definir modo de movimento: {e}")
```

```
    QMessageBox.warning(self, "Error", f"Error setting motion mode: {e}")
```

```
def on_emergency_stop_toggle(self, checked):
```

```
    """
```

```
    Manipula o clique no botão Emergency Stop/Reset.
```

```
    """
```

```
if not hasattr(self.controller.cnc, 'is_connected') or not self.controller.cnc.is_connected:
```

```
    QMessageBox.warning(self, "Erro", "CNC não conectada")
```

```
    self.emergency_stop_button.setChecked(not checked)
```

```
    return
```

```
if checked:
```

```
    # Botão foi pressionado para entrar no modo RESET
```

```

logger.info("EMERGENCY STOP: Botão pressionado. Enviando Soft Reset.")
if self.controller.cnc.send_soft_reset():
    # Configura visual do botão para Reset
    self.emergency_stop_button.setText("Reset")
    self.emergency_stop_button.setStyleSheet("background-color: orange; color: black;")

    # Atualiza status
    main_window = self.window()
    if hasattr(main_window, 'statusBar'):
        main_window.statusBar().showMessage("Máquina parada. Clique em Reset para desbloquear.")
    else:
        # Soft reset falhou, reverte o botão
        logger.error("EMERGENCY STOP: Falha ao enviar Soft Reset")
        QMessageBox.critical(self, "Erro", "Falha ao enviar comando de parada")
        self.emergency_stop_button.setChecked(False)
else:
    # Botão foi pressionado para sair do modo RESET
    logger.info("RESET: Botão pressionado para desbloquear. Enviando $X.")
    if self.controller.cnc.unlock():
        # Configura visual do botão para STOP
        self.emergency_stop_button.setText("STOP")
        self.emergency_stop_button.setStyleSheet("background-color: red; color: white;")

        # Atualiza status
        main_window = self.window()
        if hasattr(main_window, 'statusBar'):
            main_window.statusBar().showMessage("Máquina desbloqueada e pronta.")

        # Solicita atualização de status para verificar nova condição
        QTimer.singleShot(200, lambda: self.controller.cnc.grbl.send_immediately("?"))
    else:
        # Desbloqueio falhou, reverte o botão
        logger.error("RESET: Falha ao enviar comando de desbloqueio")
        QMessageBox.critical(self, "Erro", "Falha ao desbloquear a máquina")
        self.emergency_stop_button.setChecked(True)

def _auto_unlock_after_reset(self):
    """Executa sequência automática de desbloqueio após reset de emergência"""
    logger.info("AUTO UNLOCK: Iniciando sequência de desbloqueio automático após reset")

    try:
        # 1. Envia comando de desbloqueio
        self.controller.cnc.grbl.send_immediately("$X")
        logger.info("AUTO UNLOCK: Comando $X enviado")

        # 2. Pequena pausa
        QTimer.singleShot(200, lambda: self._check_if_unlocked())
    except Exception as e:
        logger.error(f"AUTO UNLOCK: Erro ao enviar comando de desbloqueio: {e}")
        main_window = self.window()

```

```

        if hasattr(main_window, 'statusBar'):
            main_window.statusBar().showMessage(f"Erro no desbloqueio automático: {e}")

def _check_if_unlocked(self):
    """Verifica se o desbloqueio foi bem-sucedido e restaura configurações se necessário"""
    try:
        # Solicita status para conferir se saiu do alarme
        self.controller.cnc.grbl.send_immediately("?")

        # Exibe mensagem de sucesso
        main_window = self.window()
        if hasattr(main_window, 'statusBar'):
            main_window.statusBar().showMessage("Sistema parado e desbloqueado automaticamente.")

        logger.info("AUTO UNLOCK: Sequência de desbloqueio automático concluída")
    except Exception as e:
        logger.error(f"AUTO UNLOCK: Erro ao verificar status após desbloqueio: {e}")

class AOIControllerApp(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Controle de Inspeção Óptica Automatizada")
        self.setGeometry(100, 100, 1200, 800)

        # Inicializa o controlador AOI
        self.controller = CNCAOIController()
        self.current_sequence = None
        self.is_running_sequence = False

        # Variável para armazenar o último valor de posição (para comparação)
        self.last_logged_position = None

        # Armazenar o offset do sistema de coordenadas de trabalho (WCS) ativo (ex: G54)
        self.current_wcs_offset = {'x': 0.0, 'y': 0.0, 'z': 0.0} # Inicializa o offset WCS padrão (G54)

        # Armazenar a última Posição da Máquina (MPos) conhecida
        self.current_mpos = {'x': 0.0, 'y': 0.0, 'z': 0.0}
        # Armazenar o Sistema de Coordenadas de Trabalho (WCS) ativo (ex: "G54")
        self.active_wcs = "G54" # Assume G54 como padrão inicial
        # Mapeamento de WCS para número P do G10
        self.wcs_to_p = {"G54": 1, "G55": 2, "G56": 3, "G57": 4, "G58": 5, "G59": 6}

        # Configuração da interface
        self.setup_ui()
        # Configuração do menu
        self.setup_menu()
        # Timer para atualizar a posição – agora conectamos a um método que loga a ação
        self.update_timer = QTimer(self)
        self.update_timer.timeout.connect(self.on_update_timer)
        self.update_timer.start(1000) # Atualiza a cada 1000ms

```

```

def on_update_timer(self):
    #logger.debug("Timer fired: atualizando tela de posição da head.")
    self.update_position_display()

def setup_ui(self):
    # Widget central
    central_widget = QWidget()
    self.setCentralWidget(central_widget)

    # Layout principal
    main_layout = QVBoxLayout(central_widget)

    # Grupo de conexão
    connection_group = QGroupBox("Conexão")
    connection_layout = QGridLayout()

    # CNC Connection
    connection_layout.addWidget(QLabel("Porta CNC:"), 0, 0)
    self.cnc_port_combo = QComboBox()
    self.refresh_ports()
    connection_layout.addWidget(self.cnc_port_combo, 0, 1)

    self.connect_cnc_btn = QPushButton("Conectar CNC")
    self.connect_cnc_btn.clicked.connect(self.connect_cnc)
    connection_layout.addWidget(self.connect_cnc_btn, 0, 2)

    # Camera connection
    connection_layout.addWidget(QLabel("Câmera ID:"), 1, 0)
    self.camera_id_combo = QComboBox()
    self.camera_id_combo.addItem("0")
    self.camera_id_combo.addItem("1")
    self.camera_id_combo.addItem("2")
    self.camera_id_combo.addItem("3")
    connection_layout.addWidget(self.camera_id_combo, 1, 1)

    self.connect_camera_btn = QPushButton("Conectar Câmera")
    self.connect_camera_btn.clicked.connect(self.connect_camera)
    connection_layout.addWidget(self.connect_camera_btn, 1, 2)

    # Refresh ports button
    self.refresh_ports_btn = QPushButton("Atualizar Portas")
    self.refresh_ports_btn.clicked.connect(self.refresh_ports)
    connection_layout.addWidget(self.refresh_ports_btn, 0, 3)

    # Test camera button
    self.test_camera_btn = QPushButton("Testar Câmera")
    self.test_camera_btn.clicked.connect(self.test_camera)
    connection_layout.addWidget(self.test_camera_btn, 1, 3)

    connection_group.setLayout(connection_layout)
    main_layout.addWidget(connection_group)

    # Grupo de Calibração de Movimento
    calibration_group = QGroupBox("Calibração de Movimento")

```

```

calibration_layout = QHBoxLayout()

calibration_layout.addWidget(QLabel("Pulsos/Revolução:"))
self.pulses_input = QLineEdit("400")
calibration_layout.addWidget(self.pulses_input)

calibration_layout.addWidget(QLabel("Passo do Fuso (mm):"))
self.fuso_input = QLineEdit("5")
calibration_layout.addWidget(self.fuso_input)

self.apply_calibration_btn = QPushButton("Aplicar Calibração")
self.apply_calibration_btn.clicked.connect(self.apply_calibration)
calibration_layout.addWidget(self.apply_calibration_btn)

calibration_group.setLayout(calibration_layout)
main_layout.addWidget(calibration_group)

# Splitter para dividir a interface em painéis
splitter = QSplitter(Qt.Orientation.Horizontal)

# Painel esquerdo: controles e lista de posições
left_panel = QWidget()
left_layout = QVBoxLayout(left_panel)

# Informações de posição
position_group = QGroupBox("Posição Atual")
position_layout = QGridLayout()

# Exibe os valores X, Y e status
position_layout.addWidget(QLabel("X:"), 0, 0)
self.x_position = QLabel("0.000 mm")
position_layout.addWidget(self.x_position, 0, 1)

position_layout.addWidget(QLabel("Y:"), 1, 0)
self.y_position = QLabel("0.000 mm")
position_layout.addWidget(self.y_position, 1, 1)

position_layout.addWidget(QLabel("Status:"), 2, 0)
self.cnc_status = QLabel("Desconectado")
position_layout.addWidget(self.cnc_status, 2, 1)

# NOVO: Botão para setar a posição atual como zero
self.set_zero_btn = QPushButton("Setar Posição Zero")
self.set_zero_btn.clicked.connect(self.set_zero_position)
position_layout.addWidget(self.set_zero_btn, 3, 0, 1, 2)

position_group.setLayout(position_layout)
left_layout.addWidget(position_group)

# Widget de lista de posições
self.position_list_widget = PositionListWidget()

```

```

self.position_list_widget.add_position_btn.clicked.connect(self.add_current_position)
self.position_list_widget.remove_position_btn.clicked.connect(self.remove_position)
self.position_list_widget.position_selected.connect(self.on_position_selected)

left_layout.addWidget(self.position_list_widget)

# Controle de sequência
self.sequence_widget = SequenceControlWidget()
self.sequence_widget.create_sequence_btn.clicked.connect(self.create_sequence)
self.sequence_widget.run_sequence_btn.clicked.connect(self.run_sequence)
self.sequence_widget.stop_sequence_btn.clicked.connect(self.stop_sequence)
self.sequence_widget.save_btn.clicked.connect(self.save_program)
self.sequence_widget.load_btn.clicked.connect(self.load_program)
self.sequence_widget.save_gcode_btn.clicked.connect(self.save_gcode)
self.sequence_widget.load_gcode_btn.clicked.connect(self.load_gcode)

left_layout.addWidget(self.sequence_widget)

# Painel direito: visualizador de imagem e resultados
right_panel = QTabWidget()

# Aba de visualização de imagem
self.image_viewer = ImageViewerWidget()
right_panel.addTab(self.image_viewer, "Visualizador de Imagem")

# Aba de resultados
results_widget = QWidget()
results_layout = QVBoxLayout(results_widget)

self.results_table = QTableWidgetItem(0, 3)
self.results_table.setHorizontalHeaderLabels(["Posição", "Timestamp", "Status"])
self.results_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)

results_layout.addWidget(QLabel("Resultados da Inspeção:"))
results_layout.addWidget(self.results_table)

right_panel.addTab(results_widget, "Resultados")

# MOVER PARA AQUI: Adicionar a aba de Câmera & Movimento (após definir right_panel)
# Tab para Camera & Movement
camera_movement_tab = QWidget()
camera_movement_layout = QHBoxLayout(camera_movement_tab)

# Left side: controls and position registry
cm_left_panel = QWidget()
cm_left_layout = QVBoxLayout(cm_left_panel)

# Movement controls
self.movement_widget = MovementControlWidget(self.controller)
cm_left_layout.addWidget(self.movement_widget)

```

```
# Position registry
self.position_registry = PositionRegistryWidget(self.controller)
self.position_registry.create_sequence_btn.clicked.connect(self.create_sequence_from_registry)
cm_left_layout.addWidget(self.position_registry)
```

```
# Right side: camera preview
self.camera_preview = CameraPreviewWidget(self.controller)
self.camera_preview.image_captured.connect(self.on_image_captured)
```

```
# Define proporções para os painéis - dar mais espaço para a visualização da câmera
camera_movement_layout.addWidget(cm_left_panel, 1) # Proporção 1
camera_movement_layout.addWidget(self.camera_preview, 3) # Proporção 3 (mais espaço)
```

```
# Agora é seguro adicionar a nova aba ao right_panel que já foi definido
right_panel.addTab(camera_movement_tab, "Câmera & Movimento")
```

```
# Adiciona painéis ao splitter
splitter.addWidget(left_panel)
splitter.addWidget(right_panel)
splitter.setSizes([400, 800])
```

```
main_layout.addWidget(splitter)
```

```
# Barra de status
self.statusBar().showMessage("Pronto para conectar")
```

```
def setup_menu(self):
```

```
    """Configura o menu da aplicação"""
    menubar = self.menuBar()
```

```
# Menu de Arquivo
file_menu = menubar.addMenu('&Arquivo')
```

```
exit_action = QAction('Sair', self)
exit_action.setShortcut('Ctrl+Q')
exit_action.triggered.connect(self.close)
file_menu.addAction(exit_action)
```

```
# Menu de Ferramentas
tools_menu = menubar.addMenu('&Ferramentas')
```

```
calibration_action = QAction('Calibração CNC', self)
calibration_action.triggered.connect(self.show_calibration_dialog)
tools_menu.addAction(calibration_action)
```

```
# Menu de Ajuda
help_menu = menubar.addMenu('&Ajuda')
```

```
about_action = QAction('Sobre', self)
about_action.triggered.connect(self.show_about_dialog)
help_menu.addAction(about_action)
```

```

def show_calibration_dialog(self):
    """Mostra um diálogo para configuração de calibração"""
    dialog = QDialog(self)
    dialog.setWindowTitle("Calibração do Sistema CNC")
    dialog.setMinimumWidth(500)

    layout = QVBoxLayout(dialog)

    # Grupo de parâmetros físicos
    param_group = QGroupBox("Parâmetros da Máquina")
    param_layout = QGridLayout(param_group)

    param_layout.addWidget(QLabel("Pulsos por Revolução:"), 0, 0)
    pulses_input = QLineEdit(self.pulses_input.text())
    param_layout.addWidget(pulses_input, 0, 1)

    param_layout.addWidget(QLabel("Passo do Fuso (mm):"), 1, 0)
    fuso_input = QLineEdit(self.fuso_input.text())
    param_layout.addWidget(fuso_input, 1, 1)

    param_layout.addWidget(QLabel("Steps/mm calculado:"), 2, 0)
    steps_mm_result = QLabel("Calculando...")
    param_layout.addWidget(steps_mm_result, 2, 1)

    # Atualiza o cálculo quando os valores mudam
    def update_calculation():
        try:
            pulses = float(pulses_input.text())
            fuso = float(fuso_input.text())
            steps_mm = pulses / fuso
            steps_mm_result.setText(f"{steps_mm:.3f} steps/mm")
        except:
            steps_mm_result.setText("Erro no cálculo")

    pulses_input.textChanged.connect(update_calculation)
    fuso_input.textChanged.connect(update_calculation)
    update_calculation() # Executa o cálculo inicial

    layout.addWidget(param_group)

    # Botões de ação
    buttons_layout = QHBoxLayout()

    apply_btn = QPushButton("Aplicar Parâmetros")
    def apply_and_close():
        self.pulses_input.setText(pulses_input.text())
        self.fuso_input.setText(fuso_input.text())
        dialog.accept()
        self.apply_calibration()
    apply_btn.clicked.connect(apply_and_close)

```

```

test_btn = QPushButton("Testar Calibração")
test_btn.clicked.connect(lambda: [dialog.accept(), self.show_calibration_test_dialog()])

cancel_btn = QPushButton("Cancelar")
cancel_btn.clicked.connect(dialog.reject)

buttons_layout.addWidget(apply_btn)
buttons_layout.addWidget(test_btn)
buttons_layout.addWidget(cancel_btn)

layout.addLayout(buttons_layout)

dialog.exec()

def show_about_dialog(self):
    """Mostra informações sobre o aplicativo"""
    QMessageBox.about(self, "Sobre HesaiVision",
        "HesaiVision v1.0\n\n"
        "Sistema de Inspeção Óptica Automatizada\n"
        "Desenvolvido para controle de CNC com GRBL\n\n"
        "© 2025 HesaiVision")

def save_gcode(self):
    """Salva a sequência atual como arquivo G-CODE"""
    if not self.current_sequence:
        QMessageBox.warning(self, "Aviso", "Crie uma sequência primeiro")
        return

    from aoi_lib.gcode_manager import GCodeManager
    gcode_manager = GCodeManager()

    filename, _ = QFileDialog.getSaveFileName(
        self, "Salvar como G-CODE", "", "Arquivos G-CODE (*.gcode *.nc *.ngc)"
    )

    if filename:
        if not filename.lower().endswith((''.gcode', '.nc', '.ngc')):
            filename += '.gcode'

        if gcode_manager.save_gcode_to_file(self.current_sequence, filename):
            self.statusBar().showMessage(f"G-CODE salvo em {filename}")
        else:
            QMessageBox.critical(self, "Erro", "Falha ao salvar o arquivo G-CODE")

def apply_calibration(self):
    """
    Aplica a calibração de movimento com base nos valores informados pelo usuário.
    Envia comandos diretos para o GRBL para configurar os steps/mm.
    """
    if not self.controller.cnc.is_connected or not self.controller.cnc.grbl:

```

```
QMessageBox.warning(self, "Erro", "CNC não conectada. Conecte primeiro.")
return
```

```
try:
```

```
    pulses = float(self.pulses_input.text())
    fuso_pass = float(self.fuso_input.text())
```

```
    # Calcula steps/mm: (pulsos por revolução) / (passo do fuso em mm)
    steps_per_mm = pulses / fuso_pass
```

```
    # Armazena o valor calculado
    self.controller.cnc.steps_to_mm_factor = fuso_pass / pulses
```

```
    # Envia comandos para configurar o GRBL
    logger.info(f"CALIBRAÇÃO: Configurando steps/mm para {steps_per_mm}")
```

```
    # Verifica se o usuário deseja realmente enviar estes valores
    reply = QMessageBox.question(
        self,
        "Confirmar Calibração",
        f"Deseja enviar os seguintes parâmetros para o GRBL?\n\n"
        f"Steps/mm eixo X: {steps_per_mm:.3f}\n"
        f"Steps/mm eixo Y: {steps_per_mm:.3f}\n\n"
        f"Isso irá alterar a configuração do controlador.",
        QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No
    )
```

```
    if reply == QMessageBox.StandardButton.Yes:
        # Comandos para configurar os eixos X e Y
        self.controller.cnc.grbl.send_immediately(f"$100={steps_per_mm:.3f}")
        QTimer.singleShot(100, lambda: self.controller.cnc.grbl.send_immediately(f"$101={steps_per_mm:.3f}"))
```

```
        # Solicita ao usuário que faça um teste de calibração
        QTimer.singleShot(500, self.show_calibration_test_dialog)
```

```
        self.statusBar().showMessage(f"Calibração aplicada: {steps_per_mm:.3f} steps/mm")
    else:
        self.statusBar().showMessage("Calibração cancelada pelo usuário")
```

```
except ValueError:
    QMessageBox.warning(self, "Erro", "Valores de calibração inválidos.")
```

```
def show_calibration_test_dialog(self):
    """
```

```
    Exibe um diálogo para testar a calibração atual.
    """
```

```
    dialog = QDialog(self)
    dialog.setWindowTitle("Teste de Calibração")
    dialog.setMinimumWidth(400)
```

```
    layout = QVBoxLayout(dialog)
```

```

# Instruções
instructions = QLabel(
    "Para testar a calibração:\n\n"
    "1. Coloque um papel milimetrado ou uma régua sob a cabeça da máquina\n"
    "2. Escolha uma distância de teste\n"
    "3. Clique em 'Mover X' ou 'Mover Y' para testar cada eixo\n"
    "4. Verifique se o deslocamento físico corresponde ao valor escolhido\n"
    "5. Se necessário, ajuste os valores de calibração e aplique novamente"
)
instructions.setWordWrap(True)
layout.addWidget(instructions)

# Distância de teste
test_layout = QHBoxLayout()
test_layout.addWidget(QLabel("Distância de teste:"))
distance_input = QLineEdit("10")
test_layout.addWidget(distance_input)
test_layout.addWidget(QLabel("mm"))
layout.addLayout(test_layout)

# Botões de teste
buttons_layout = QHBoxLayout()

move_x_btn = QPushButton("Mover X")
move_x_btn.clicked.connect(lambda: self.test_calibration_move(0, float(distance_input.text())))

move_y_btn = QPushButton("Mover Y")
move_y_btn.clicked.connect(lambda: self.test_calibration_move(1, float(distance_input.text())))

reset_position_btn = QPushButton("Zerar Posição")
reset_position_btn.clicked.connect(self.set_zero_position)

buttons_layout.addWidget(move_x_btn)
buttons_layout.addWidget(move_y_btn)
buttons_layout.addWidget(reset_position_btn)

layout.addLayout(buttons_layout)

# Resultados
result_group = QGroupBox("Resultados")
result_layout = QVBoxLayout(result_group)

self.calibration_test_result = QLabel("Execute um teste para ver os resultados")
result_layout.addWidget(self.calibration_test_result)

layout.addWidget(result_group)

# Botões de controle
control_layout = QHBoxLayout()
close_btn = QPushButton("Concluir")

```

```
close_btn.clicked.connect(dialog.accept)
control_layout.addWidget(close_btn)
```

```
layout.addLayout(control_layout)
```

```
dialog.exec()
```

```
def test_calibration_move(self, axis, distance):
```

```
    """
```

```
    Realiza um movimento de teste para calibração.
```

```
    Args:
```

```
        axis: 0 para X, 1 para Y
```

```
        distance: Distância em mm para mover
```

```
    """
```

```
    if not self.controller.cnc.is_connected:
```

```
        QMessageBox.warning(self, "Erro", "CNC não conectada")
```

```
        return
```

```
    try:
```

```
        # Captura posição inicial
```

```
        initial_position = self.controller.cnc.get_current_position()
```

```
        # Prepara o comando
```

```
        axis_name = "X" if axis == 0 else "Y"
```

```
        # Define o modo absoluto para garantir precisão
```

```
        self.controller.cnc.grbl.send_immediately("G90")
```

```
        # Calcula a posição absoluta a ser atingida
```

```
        target_position = {}
```

```
        target_position['x'] = initial_position['x'] + distance if axis == 0 else initial_position['x']
```

```
        target_position['y'] = initial_position['y'] + distance if axis == 1 else initial_position['y']
```

```
        # Envia o movimento como coordenada absoluta
```

```
        cmd = f"G1 {axis_name}{target_position[axis_name.lower()]} F500"
```

```
        logger.info(f"CALIBRAÇÃO: Enviando comando de teste: {cmd}")
```

```
        self.controller.cnc.grbl.send_immediately(cmd)
```

```
        # Aguarda um pouco para o movimento ser concluído
```

```
        QTimer.singleShot(1500, lambda: self.verify_calibration_result(axis, initial_position, distance))
```

```
    except Exception as e:
```

```
        logger.error(f"CALIBRAÇÃO: Erro no teste de calibração: {e}")
```

```
        QMessageBox.warning(self, "Erro", f"Erro no teste: {str(e)}")
```

```
def verify_calibration_result(self, axis, initial_position, expected_distance):
```

```
    """
```

```
    Verifica o resultado do teste de calibração.
```

```
    Args:
```

axis: 0 para X, 1 para Y
initial_position: Posição antes do movimento
expected_distance: Distância esperada do movimento

"""

try:

Força atualização da posição

self.controller.cnc.grbl.send_immediately("?")

Aguarda um pouco para receber a atualização

time.sleep(0.2)

Obtem posição atual

current_position = self.controller.cnc.get_current_position()

Calcula o deslocamento real

axis_name = "x" if axis == 0 else "y"

actual_distance = current_position[axis_name] - initial_position[axis_name]

Calcula o erro

error = actual_distance - expected_distance

error_percent = (error / expected_distance) * 100

Atualiza o resultado do teste

result_text = (

f'Eixo: {axis_name.upper()}\n"

f'Movimento comandado: {expected_distance:.3f} mm\n"

f'Movimento real: {actual_distance:.3f} mm\n"

f'Erro: {error:.3f} mm ({error_percent:.2f}%)\n\n"

)

if abs(error_percent) < 1:

result_text += "■ Calibração excelente (erro < 1%)"

elif abs(error_percent) < 5:

result_text += "✓ Calibração aceitável (erro < 5%)"

else:

result_text += "■ Calibração insatisfatória - Ajuste os parâmetros"

if hasattr(self, 'calibration_test_result'):

self.calibration_test_result.setText(result_text)

logger.info(f"CALIBRAÇÃO: Resultado do teste - {result_text}")

except Exception as e:

logger.error(f"CALIBRAÇÃO: Erro ao verificar resultado: {e}")

def set_zero_position(self):

"""Define a posição de trabalho atual como zero usando G10 L20."""

if not self.controller.cnc.is_connected or not self.controller.cnc.grbl:

QMessageBox.warning(self, "Aviso", "CNC não conectada")

return

try:

```

# 1. Captura a MPos MAIS RECENTE armazenada ANTES de enviar o G10
# Garante que estamos usando a posição correta para calcular o novo offset.
# Usamos self.current_mpos que é atualizado pelo callback on_stateupdate.
mpos_correcta_no_zeramento = self.current_mpos.copy() # Captura a MPos atual armazenada
logger.info(f"SET ZERO: MPos capturada para zeramento: {mpos_correcta_no_zeramento}")

# Verifica se a MPos capturada parece válida (não apenas zeros se esperamos algo diferente)
# Este é um check adicional, pode ser ajustado ou removido se causar problemas.
if mpos_correcta_no_zeramento['x'] == 0.0 and mpos_correcta_no_zeramento['y'] == 0.0 and (self.last_logged_posi
    logger.warning(f"SET ZERO: MPos capturada ({mpos_correcta_no_zeramento}) parece zerada, mas a última po
# Poderia até abortar aqui ou pedir confirmação, mas vamos prosseguir por enquanto.

# 2. Obter o número P correspondente ao WCS ativo (Ex: G54 -> P1)
p_number = self.wcs_to_p.get(self.active_wcs)
if p_number is None:
    logger.error(f"SET ZERO: WCS ativo '{self.active_wcs}' não reconhecido para G10 L20. Usando P1 (G54).")
    p_number = 1 # Usa G54 como fallback

# 3. Construir o comando G10 L20
# Zerando apenas X e Y por enquanto. Adicione Z se necessário: Z{mpos_correcta_no_zeramento['z']:.4f}
# O comando G10 L20 Pn X0 Y0 diz ao GRBL: "Ajuste o offset do WCS 'n' para que a MPos *atual* corresponda a
command = f"G10 L20 P{p_number} X0 Y0"
logger.info(f"SET ZERO: Enviando comando: {command} para zerar {self.active_wcs}")

# 4. Enviar o comando G10 L20 para o GRBL
self.controller.cnc.grbl.send_immediately(command)

# 5. Atualizar o offset interno da APLICAÇÃO
# O novo offset que a aplicação deve usar para calcular WPos = MPos - Offset
# é exatamente a MPos que a máquina tinha no momento do comando G10.
logger.info(f"SET ZERO: Atualizando offset interno de {self.current_wcs_offset} para {mpos_correcta_no_zeramento}
self.current_wcs_offset = mpos_correcta_no_zeramento # ATUALIZAÇÃO CORRETA DO OFFSET INTERNO

# 6. Atualizar a posição interna da APLICAÇÃO (WPos) para zero
# Isso força a exibição a mostrar (0,0) imediatamente.
new_wpos = {'x': 0.0, 'y': 0.0, 'z': 0.0} # A WPos deve ser zero agora
logger.info(f"SET ZERO: Forçando posição interna (WPos) para {new_wpos}")
self.controller.cnc.current_position = new_wpos

# 7. Atualizar a interface gráfica imediatamente com a WPos zerada
self.update_position_display() # Chama a função que atualiza os labels X e Y

self.statusBar().showMessage(f"Posição zero definida para {self.active_wcs} na localização atual.")

# 8. (Opcional) Solicitar $# ou $G após um tempo para verificar se o GRBL processou
QTimer.singleShot(500, lambda: self.controller.cnc.grbl.send_immediately("$#"))
QTimer.singleShot(600, lambda: self.controller.cnc.grbl.send_immediately("$G"))

except Exception as e:
    logger.error(f"SET ZERO: Erro ao definir posição zero: {e}", exc_info=True)
    QMessageBox.warning(self, "Erro", f"Falha ao setar posição zero: {str(e)}")

```

```

def on_image_captured(self, image, position_name):
    """Handle captured image and register position"""
    # Display in image viewer
    self.image_viewer.display_image(image, f"Image: {position_name}")

    # Get current position
    if not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Error", "CNC not connected")
        return

    current_pos = self.controller.cnc.get_current_position()

    # Register position with image
    self.position_registry.add_position(
        position_name,
        current_pos['x'],
        current_pos['y'],
        image
    )

    self.statusBar().showMessage(f"Position '{position_name}' registered at X:{current_pos['x']:.3f}, Y:{current_pos['y']:.3f}")

def create_sequence_from_registry(self):
    """Create a sequence from registered positions"""
    if not self.position_registry.positions:
        QMessageBox.warning(self, "Warning", "No positions registered")
        return

    # Get sequence name
    sequence_name = self.sequence_widget.sequence_name.text()
    if not sequence_name:
        QMessageBox.warning(self, "Warning", "Please enter a sequence name")
        return

    # Clear existing positions in the position list widget
    self.position_list_widget.clear_positions()

    # Create positions for the sequence
    positions = []
    for pos in self.position_registry.positions:
        # Create position object with camera parameters
        camera_params = {"has_image": pos['image'] is not None}

        inspection_pos = InspectionPosition(
            pos['name'],
            pos['x'],
            pos['y'],
            camera_params
        )
        positions.append(inspection_pos)

```

```

        # Also add to the position list widget
        self.position_list_widget.add_position(inspection_pos)

# Create the sequence
self.current_sequence = self.controller.create_sequence(sequence_name, positions)

# Update UI
self.sequence_widget.sequence_status.setText(f"Created: {len(positions)} positions")
self.statusBar().showMessage(f'Sequence '{sequence_name}' created with {len(positions)} positions')

# Ask if user wants to save as G-CODE
reply = QMessageBox.question(
    self,
    "Save G-CODE",
    "Do you want to save this sequence as G-CODE?",
    QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No
)

if reply == QMessageBox.StandardButton.Yes:
    self.save_gcode()

def load_gcode(self):
    """Carrega uma sequência a partir de um arquivo G-CODE"""
    from aoj_lib.gcode_manager import GCodeManager
    gcode_manager = GCodeManager()

    filename, _ = QFileDialog.getOpenFileName(
        self, "Abrir G-CODE", "", "Arquivos G-CODE (*.gcode *.nc *.ngc)"
    )

    if filename:
        # Carrega o G-CODE
        result = gcode_manager.read_gcode_file(filename)

        if result:
            # Limpa posições atuais
            self.position_list_widget.clear_positions()

            # Cria sequência a partir dos dados do G-CODE
            positions = []
            for pos_data in result['positions']:
                # Cria objeto de posição
                position = InspectionPosition(
                    pos_data['name'],
                    pos_data['x'],
                    pos_data['y'],
                    pos_data['camera_params']
                )

                # Adiciona à lista visual

```

```
self.position_list_widget.add_position(position)
```

```
# Adiciona à lista interna  
positions.append(position)
```

```
# Cria a sequência  
self.current_sequence = self.controller.create_sequence(  
    result['sequence_name'], positions  
)
```

```
# Atualiza interface  
self.sequence_widget.sequence_name.setText(result['sequence_name'])  
self.sequence_widget.sequence_status.setText(  
    f"Carregada do G-CODE: {len(positions)} posições"  
)
```

```
self.statusBar().showMessage(f"G-CODE carregado de {filename}")  
else:  
    QMessageBox.critical(self, "Erro", "Falha ao carregar o arquivo G-CODE")
```

```
def refresh_ports(self):
```

```
    """Atualiza a lista de portas seriais disponíveis"""
```

```
    import serial.tools.list_ports
```

```
    self.cnc_port_combo.clear()
```

```
    ports = [port.device for port in serial.tools.list_ports.comports()]
```

```
    if ports:
```

```
        self.cnc_port_combo.addItems(ports)
```

```
    # Verificar se COM9 está na lista e selecionar
```

```
    com9_index = self.cnc_port_combo.findText("COM9")
```

```
    if com9_index >= 0:
```

```
        self.cnc_port_combo.setCurrentIndex(com9_index)
```

```
        self.statusBar().showMessage("Porta COM9 detectada")
```

```
    else:
```

```
        self.statusBar().showMessage("Nenhuma porta serial encontrada")
```

```
def get_current_position(self):
```

```
    """
```

```
    Retorna a posição atual do CNC.
```

```
    Certifica-se de retornar uma cópia do dicionário para evitar alteração acidental.
```

```
    """
```

```
    logger.debug(f"get_current_position chamado, retornando: {self.current_position}")
```

```
    return self.current_position.copy() if hasattr(self, 'current_position') else {'x': 0, 'y': 0, 'z': 0}
```

```
def connect_cnc(self):
```

```
    """Conecta à máquina CNC usando a biblioteca grbl-streamer"""
```

```
    if hasattr(self.controller.cnc, 'is_connected') and self.controller.cnc.is_connected:
```

```
        # Desconectar
```

```
        try:
```

```

if hasattr(self.controller.cnc, 'grbl') and self.controller.cnc.grbl:
    self.controller.cnc.grbl.poll_stop()
    self.controller.cnc.grbl.disconnect()
    self.controller.cnc.grbl = None
    self.controller.cnc.is_connected = False
    self.connect_cnc_btn.setText("Conectar CNC")
    self.cnc_status.setText("Desconectado")
    self.statusBar().showMessage("CNC desconectada")
except Exception as e:
    self.statusBar().showMessage(f"Erro ao desconectar: {str(e)}")
else:
    # Conectar
    port = self.cnc_port_combo.currentText()
    if not port:
        QMessageBox.warning(self, "Erro", "Selecione uma porta serial")
        return

    self.statusBar().showMessage(f"Conectando à CNC na porta {port}...")

    try:
        # Define função de callback para eventos do GrblStreamer
        def grbl_callback(eventstring, *data):
            logger.debug(f"CALLBACK: Evento '{eventstring}' recebido com data: {data}")

            # Capturar offsets do sistema de coordenadas (G54, G55, etc.)
            if eventstring == "on_hash_stateupdate":
                if data and isinstance(data[0], dict):
                    hash_state = data[0]

                    # Processar G54 APENAS se o WCS ativo for G54
                    # Isso evita que a resposta tardia do $$ sobrescreva um offset
                    # que foi definido manualmente via G10 L20 para G54.
                    if self.active_wcs == "G54":
                        g54_offset_data = hash_state.get('G54')
                        if isinstance(g54_offset_data, (list, tuple)) and len(g54_offset_data) >= 2:
                            try:
                                new_offset_x = float(g54_offset_data[0])
                                new_offset_y = float(g54_offset_data[1])
                                new_offset_z = float(g54_offset_data[2]) if len(g54_offset_data) > 2 else 0.0

                                # Atualiza o offset interno
                                self.current_wcs_offset['x'] = new_offset_x
                                self.current_wcs_offset['y'] = new_offset_y
                                self.current_wcs_offset['z'] = new_offset_z
                                logger.info(f"CALLBACK: Offset G54 (de $$) atualizado para: {self.current_wcs_offset} (WCS ativo)")
                            except (ValueError, TypeError):
                                logger.error(f"CALLBACK: Erro ao converter offset G54 de $$: {g54_offset_data}")
                        else:
                            logger.warning(f"CALLBACK: Offset G54 não encontrado ou inválido nos dados hash para WCS ativo")
                    else:
                        logger.debug(f"CALLBACK: Ignorando atualização de offset G54 de $$ porque WCS ativo é {self.active_wcs}")
    
```

TODO: Se precisar suportar outros WCS (G55-G59), adicionar lógica similar aqui

Capturar estado do parser para saber o WCS ativo

elif eventstring == "on_gcode_parser_stateupdate":

... (código existente para atualizar self.active_wcs) ...

if data and isinstance(data[0], list) and len(data[0]) > 1:

 parser_state = data[0]

 # O índice 1 contém o WCS ativo (ex: "54", "55", etc.)

 # O índice 0 contém o modo de movimento (ex: "1" para G1)

 # O índice 4 contém o modo de distância (ex: "90" para G90)

 new_active_wcs = f"G{parser_state[1]}"

 new_motion_mode = f"G{parser_state[0]}"

 new_distance_mode = f"G{parser_state[4]}"

if new_active_wcs != self.active_wcs:

 logger.info(f"CALLBACK: WCS Ativo mudou de {self.active_wcs} para {new_active_wcs}")

 self.active_wcs = new_active_wcs

 # Ao mudar o WCS, seria ideal buscar o offset correspondente via \$\$

 # ou ter todos os offsets armazenados. Por enquanto, apenas logamos.

 # self.controller.cnc.grbl.send_immediately("\$#") # Cuidado com loops

Atualiza estado interno da aplicação sobre modos G90/G91

Isso garante que a UI e a lógica de movimento estejam sincronizadas

com o estado real do GRBL reportado por \$G.

if hasattr(self, 'movement_widget'): # Verifica se o widget existe

 if new_distance_mode == "G90":

 if not self.movement_widget.mode_absolute.isChecked():

 logger.info("CALLBACK (\$G): Sincronizando UI para G90 (Absoluto)")

 self.movement_widget.mode_absolute.setChecked(True)

 self.movement_widget.mode_relative.setChecked(False)

 elif new_distance_mode == "G91":

 if not self.movement_widget.mode_relative.isChecked():

 logger.info("CALLBACK (\$G): Sincronizando UI para G91 (Relativo)")

 self.movement_widget.mode_absolute.setChecked(False)

 self.movement_widget.mode_relative.setChecked(True)

elif eventstring == "on_stateupdate":

 logger.info(f"CALLBACK: Processando 'on_stateupdate'. Dados brutos: {data}")

if len(data) >= 3:

 state = data[0]

 mpos_tuple = data[1] # Posição da Máquina (MPos)

 # wpos_tuple = data[2] # Posição de Trabalho (WPos) - Ignorando pois está vindo zerado

 logger.debug(f"CALLBACK DETALHADO: state={state}, mpos={mpos_tuple}") # Removido wpos do log d

Atualiza estado da máquina

old_state = self.controller.cnc.machine_status if hasattr(self.controller.cnc, 'machine_status') else None

self.controller.cnc.machine_status = state

if old_state != state:

```

logger.debug(f"CALLBACK: Estado da máquina mudou de '{old_state}' para '{state}'")

# Calcular WPOS a partir de MPOS e do offset armazenado
if isinstance(mpos_tuple, (list, tuple)) and len(mpos_tuple) >= 2:
    try:
        # Posição da Máquina recebida
        mpos_x = float(mpos_tuple[0])
        mpos_y = float(mpos_tuple[1])
        mpos_z = float(mpos_tuple[2]) if len(mpos_tuple) > 2 else 0.0

        # Calcular Posição de Trabalho (WPos = MPos - Offset)
        calculated_wpos_x = mpos_x - self.current_wcs_offset['x']
        calculated_wpos_y = mpos_y - self.current_wcs_offset['y']
        calculated_wpos_z = mpos_z - self.current_wcs_offset['z']

        # Usar a WPos calculada para atualizar a posição interna
        new_position = {
            'x': calculated_wpos_x,
            'y': calculated_wpos_y,
            'z': calculated_wpos_z
        }

        old_position = None
        if hasattr(self.controller.cnc, 'current_position'):
            old_position = self.controller.cnc.current_position.copy()
            # logger.debug(f"CALLBACK: Posição interna ANTES da atualização: {old_position}") # Log opcional

        logger.debug(f"CALLBACK: MPos={mpos_tuple}, Offset={self.current_wcs_offset}, WPos Calculada={calculated_wpos}")
        logger.debug(f"CALLBACK: Tentando atualizar posição interna (usando WPOS CALCULADA) para: {new_position}")

        # Compara new_position (WPos calculada) com old_position
        position_changed = (old_position is None) or \
            (abs(old_position['x'] - new_position['x']) > 1e-4) or \
            (abs(old_position['y'] - new_position['y']) > 1e-4) or \
            (abs(old_position.get('z', 0.0) - new_position.get('z', 0.0)) > 1e-4)

        if position_changed:
            logger.info(f"CALLBACK: POSIÇÃO INTERNA ATUALIZADA (usando WPOS CALCULADA): {old_position} para {new_position}")
            # Atualiza a posição no controlador com a WPos calculada
            self.controller.cnc.current_position = new_position

    except (ValueError, TypeError, IndexError) as e:
        logger.error(f"CALLBACK: Erro ao processar MPOS ou calcular WPOS: {e}, mpos={mpos_tuple}")
    else:
        logger.error(f"CALLBACK: Formato inválido para MPOS: {type(mpos_tuple)}, valor: {mpos_tuple}")

else:
    logger.error(f"CALLBACK: 'on_stateupdate' recebido com dados insuficientes (len={len(data)}). Dados: {data}")

# ... (restante do código do callback para outros eventos: on_write, on_read, etc.) ...
elif eventstring == "on_write":

```

```

        logger.debug(f"CALLBACK: Comando enviado para GRBL: {data[0] if data else 'vazio'}")
        # ... (etc.) ...

# --- INÍCIO DA MODIFICAÇÃO ---
# Inicializa o GrblStreamer APENAS com o callback
self.controller.cnc.grbl = GrblStreamer(grbl_callback)
# --- FIM DA MODIFICAÇÃO ---

logger.debug(f"CONEXÃO: Tentando conectar à porta {port} com baudrate 115200")
# Conecta usando o método cnect()
self.controller.cnc.grbl.cnect(port, 115200)
time.sleep(2.0)

if not self.controller.cnc.grbl.connected:
    logger.error("CONEXÃO: Falha ao estabelecer conexão serial (grbl.connected é False).")
    raise ConnectionError("Falha ao conectar à porta serial após inicialização.")

logger.debug("CONEXÃO: Enviando comando de desbloqueio $X")
self.controller.cnc.grbl.send_immediately("$X")
time.sleep(0.1)

logger.debug("CONEXÃO: Configurando $10=3 para relatório completo de posição")
self.controller.cnc.grbl.send_immediately("$10=3") # Mantém $10=3 para receber MPos
time.sleep(0.1)

# --- INÍCIO DA MODIFICAÇÃO ---
# Solicitar estado hash logo após conectar para obter offsets
logger.debug("CONEXÃO: Solicitando estado hash ($#) para obter offsets")
self.controller.cnc.grbl.send_immediately("#")
time.sleep(0.1)
# --- FIM DA MODIFICAÇÃO ---

logger.debug("CONEXÃO: Configurando modo relativo G91")
self.controller.cnc.grbl.send_immediately("G91")
time.sleep(0.1)

logger.debug("CONEXÃO: Iniciando polling de status")
self.controller.cnc.grbl.poll_start()

self.controller.cnc.is_connected = True
self.controller.cnc.machine_status = "Idle"
self.connect_cnc_btn.setText("Desconectar CNC")
self.cnc_status.setText("Conectado")
self.statusBar().showMessage(f"CNC conectada na porta {port}")

except Exception as e:
    # ... (código de tratamento de erro de conexão) ...
    logger.error(f"CONEXÃO: Falha ao conectar ou configurar: {str(e)}", exc_info=True)
    QMessageBox.critical(self, "Erro", f"Falha ao conectar a CNC: {str(e)}")
    if hasattr(self.controller.cnc, 'grbl') and self.controller.cnc.grbl:
        try:

```

```
        self.controller.cnc.grbl.disconnect()
    except: pass
    self.controller.cnc.grbl = None
    self.controller.cnc.is_connected = False
    self.controller.cnc.machine_status = "Erro Conexão"
    self.connect_cnc_btn.setText("Conectar CNC")
    self.cnc_status.setText("Erro Conexão")
```

```
def connect_camera(self):
```

```
    """Conecta à câmera"""
```

```
    if hasattr(self.controller.camera, 'is_connected') and self.controller.camera.is_connected:
```

```
        # Desconectar
```

```
        self.controller.camera.disconnect()
```

```
        self.connect_camera_btn.setText("Conectar Câmera")
```

```
        self.statusBar().showMessage("Câmera desconectada")
```

```
    else:
```

```
        # Conectar
```

```
        try:
```

```
            camera_id = int(self.camera_id_combo.currentText())
```

```
            self.statusBar().showMessage(f"Conectando à câmera ID {camera_id}...")
```

```
            if self.controller.connect_camera(camera_id):
```

```
                self.connect_camera_btn.setText("Desconectar Câmera")
```

```
                self.statusBar().showMessage(f"Câmera ID {camera_id} conectada")
```

```
            else:
```

```
                QMessageBox.critical(self, "Erro", f"Falha ao conectar à câmera: {self.controller.camera.last_error}")
```

```
        except ValueError:
```

```
            QMessageBox.warning(self, "Erro", "ID de câmera inválido")
```

```
def test_camera(self):
```

```
    """Testa a captura de imagem da câmera"""
```

```
    if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
```

```
        QMessageBox.warning(self, "Aviso", "Câmera não conectada")
```

```
    return
```

```
    image = self.controller.camera.capture()
```

```
    if image is not None:
```

```
        self.image_viewer.display_image(image, "Imagem de teste da câmera")
```

```
        self.statusBar().showMessage("Imagem capturada com sucesso")
```

```
    else:
```

```
        QMessageBox.warning(self, "Erro", f"Falha ao capturar imagem: {self.controller.camera.last_error}")
```

```
def update_position_display(self):
```

```
    """Atualiza a exibição da posição atual (agora exibindo WPos calculada)"""
```

```
    if not self.controller.cnc.is_connected:
```

```
        # logger.debug("update_position_display: CNC não conectada.") # Log já existente
```

```
        return
```

```
    try:
```

```
        # get_current_position agora retorna a WPos calculada
```

```

        position = self.controller.cnc.get_current_position()
        logger.debug("update_position_display: posição (WPos calculada) obtida do CNC: %s", position)
except Exception as e:
    logger.error("update_position_display: erro ao obter posição: %s", e)
    return

# Comparação para log (opcional, pode ser removido se poluir muito)
if self.last_logged_position is not None:
    if position == self.last_logged_position:
        pass
        # logger.warning("update_position_display: posição (WPos calculada) inalterada: %s", position)
    else:
        logger.debug("update_position_display: posição (WPos calculada) mudou de %s para %s",
                      self.last_logged_position, position)
else:
    logger.debug("update_position_display: nenhuma posição (WPos calculada) anterior registrada.")

self.last_logged_position = position.copy()

# Atualiza os labels da interface com a WPos calculada
self.x_position.setText(f'{position["x"]:.3f} mm')
self.y_position.setText(f'{position["y"]:.3f} mm')
self.cnc_status.setText(self.controller.cnc.machine_status)

def add_current_position(self):
    """Adiciona a posição atual à lista"""
    if not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Aviso", "CNC não conectada")
        return

    position_name, ok = QInputDialog.getText(self, "Nova Posição", "Nome da posição:")

    if ok and position_name:
        position = self.controller.add_current_position(position_name)
        self.position_list_widget.add_position(position)
        self.statusBar().showMessage(f'Posição '{position_name}' adicionada')

def remove_position(self):
    """Remove a posição selecionada"""
    position = self.position_list_widget.remove_selected_position()

    if position:
        self.controller.position_manager.remove_position(position.name)
        self.statusBar().showMessage(f'Posição '{position.name}' removida')

def on_position_selected(self, position):
    """Manipula a seleção de uma posição"""
    # Poderia mover para esta posição, mostrar detalhes, etc.
    self.statusBar().showMessage(f'Posição selecionada: {position.name} ({position.x:.3f}, {position.y:.3f})')

def create_sequence(self):

```

```
"""Cria uma nova sequência com as posições atuais"""
```

```
positions = self.position_list_widget.get_all_positions()
```

```
if not positions:
```

```
    QMessageBox.warning(self, "Aviso", "Adicione pelo menos uma posição")
```

```
    return
```

```
sequence_name = self.sequence_widget.sequence_name.text()
```

```
if not sequence_name:
```

```
    QMessageBox.warning(self, "Aviso", "Digite um nome para a sequência")
```

```
    return
```

```
self.current_sequence = self.controller.create_sequence(sequence_name, positions)
```

```
self.statusBar().showMessage(f'Sequência '{sequence_name}' criada com {len(positions)} posições')
```

```
self.sequence_widget.sequence_status.setText(f'Criada: {len(positions)} posições')
```

```
def run_sequence(self):
```

```
    """Executes the current sequence"""
```

```
    if not self.current_sequence:
```

```
        QMessageBox.warning(self, "Warning", "Create a sequence first")
```

```
        return
```

```
    if not self.controller.cnc.is_connected:
```

```
        QMessageBox.warning(self, "Warning", "CNC not connected")
```

```
        return
```

```
    if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
```

```
        QMessageBox.warning(self, "Warning", "Camera not connected")
```

```
        return
```

```
# Clear previous results
```

```
self.results_table.setRowCount(0)
```

```
# Configure UI for execution
```

```
self.is_running_sequence = True
```

```
self.sequence_widget.run_sequence_btn.setEnabled(False)
```

```
self.sequence_widget.stop_sequence_btn.setEnabled(True)
```

```
self.sequence_widget.sequence_status.setText("Executing...")
```

```
# Start execution
```

```
try:
```

```
    self.statusBar().showMessage(f'Executing sequence '{self.current_sequence.name}'...')
```

```
# Use a thread to run the sequence
```

```
self.run_thread = SequenceRunnerThread(self.controller, self.current_sequence.name)
```

```
self.run_thread.image_captured.connect(self.on_sequence_image_captured)
```

```
self.run_thread.sequence_completed.connect(self.on_sequence_completed)
```

```
self.run_thread.sequence_error.connect(self.on_sequence_error)
```

```
self.run_thread.start()
```

```
except Exception as e:
```

```
self.statusBar().showMessage(f"Error executing sequence: {str(e)}")
self.sequence_widget.sequence_status.setText("Error")
self.on_sequence_completed()
```

```
def on_sequence_image_captured(self, result):
```

```
    """Called when an image is captured during sequence execution"""
```

```
    position = result["position"]
```

```
    image = result["image"]
```

```
    timestamp = result["timestamp"]
```

```
    # Display the image
```

```
    self.image_viewer.display_image(image, f"Position: {position.name} ({position.x:.3f}, {position.y:.3f})")
```

```
    # Add to results table
```

```
    row = self.results_table.rowCount()
```

```
    self.results_table.insertRow(row)
```

```
    self.results_table.setItem(row, 0, QTableWidgetItem(position.name))
```

```
    self.results_table.setItem(row, 1, QTableWidgetItem(time.strftime("%H:%M:%S", time.localtime(timestamp))))
```

```
    self.results_table.setItem(row, 2, QTableWidgetItem("Captured"))
```

```
def stop_sequence(self):
```

```
    """Para a execução da sequência atual"""
```

```
    if self.is_running_sequence:
```

```
        self.controller.stop_sequence()
```

```
        self.statusBar().showMessage("Execução de sequência interrompida")
```

```
        self.on_sequence_completed()
```

```
def on_sequence_completed(self):
```

```
    """Chamado quando a execução da sequência é concluída"""
```

```
    self.is_running_sequence = False
```

```
    self.sequence_widget.run_sequence_btn.setEnabled(True)
```

```
    self.sequence_widget.stop_sequence_btn.setEnabled(False)
```

```
    self.sequence_widget.sequence_status.setText("Concluída")
```

```
    self.statusBar().showMessage("Execução de sequência concluída")
```

```
def on_sequence_error(self, error_message):
```

```
    """Chamado quando ocorre um erro durante a execução da sequência"""
```

```
    QMessageBox.critical(self, "Erro na Sequência", error_message)
```

```
    self.on_sequence_completed()
```

```
def save_program(self):
```

```
    """Salva o programa de inspeção atual"""
```

```
    if not self.current_sequence:
```

```
        QMessageBox.warning(self, "Aviso", "Crie uma sequência primeiro")
```

```
        return
```

```
    filename, _ = QFileDialog.getSaveFileName(self, "Salvar Programa", "", "Arquivos JSON (*.json)")
```

```
    if filename:
```

```
        if not filename.endswith('.json'):
```

```
            filename += '.json'
```

```

if self.controller.save_positions(filename):
    self.statusBar().showMessage(f"Programa salvo em {filename}")
else:
    QMessageBox.critical(self, "Erro", "Falha ao salvar o programa")

```

```

def load_program(self):
    """Carrega um programa de inspeção salvo"""
    filename, _ = QFileDialog.getOpenFileName(self, "Carregar Programa", "", "Arquivos JSON (*.json)")

```

```

if filename:
    if self.controller.load_positions(filename):
        # Atualiza a interface
        self.position_list_widget.clear_positions()

        # Adiciona posições carregadas à lista
        for name, position in self.controller.position_manager.positions.items():
            self.position_list_widget.add_position(position)

        # Se há sequências, usa a primeira como atual
        if self.controller.position_manager.sequences:
            sequence_name = next(iter(self.controller.position_manager.sequences))
            self.current_sequence = self.controller.position_manager.sequences[sequence_name]
            self.sequence_widget.sequence_name.setText(sequence_name)
            self.sequence_widget.sequence_status.setText(f"Carregada: {len(self.current_sequence.positions)} posições")

            self.statusBar().showMessage(f"Programa carregado de {filename}")
        else:
            QMessageBox.critical(self, "Erro", "Falha ao carregar o programa")

```

```

def closeEvent(self, event):
    """Manipula o evento de fechamento da janela"""
    # Para a execução da sequência, se houver
    if self.is_running_sequence:
        self.controller.stop_sequence()

    # Desconecta da câmera e CNC
    if hasattr(self.controller.camera, 'is_connected') and self.controller.camera.is_connected:
        self.controller.camera.disconnect()

    if self.controller.cnc.is_connected:
        self.controller.cnc.disconnect()

    event.accept()

```

```

class SequenceRunnerThread(QThread):
    """Thread para executar uma sequência de inspeção"""
    image_captured = pyqtSignal(dict) # Emite resultados da captura
    sequence_completed = pyqtSignal() # Emite quando a sequência é concluída
    sequence_error = pyqtSignal(str) # Emite quando ocorre um erro

```

```
def __init__(self, controller, sequence_name):
    super().__init__()
    self.controller = controller
    self.sequence_name = sequence_name

def run(self):
    try:
        # Execute a sequência e processe os resultados
        def process_result(result):
            self.image_captured.emit(result)

        self.controller.run_sequence(self.sequence_name, process_result)
        self.sequence_completed.emit()

    except Exception as e:
        self.sequence_error.emit(str(e))

# Classe QDialog que estava faltando
from PyQt6.QtWidgets import QDialog

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = AOIControllerApp()
    window.show()
    sys.exit(app.exec())
```